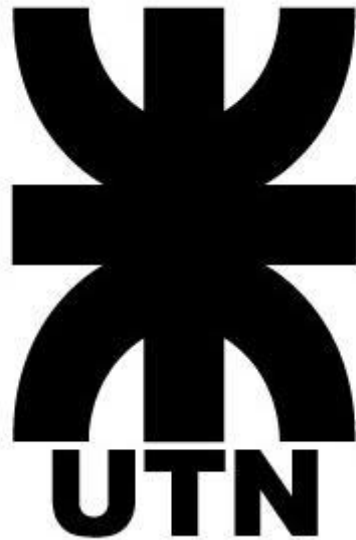


Universidad Tecnológica Nacional



Cátedra: Algoritmos Genéticos

**4º Año Ingeniería en Sistemas de Información
Trabajo Práctico N°1**

Comisión: 303

Grupo : 14

Integrantes:

- RIBOTTA, Tomás	52309	tomasribotta20@gmail.com
- MAZALAN, Ariel	52867	arielmazalan11@gmail.com
- GIACONE, Alessandro	52664	alef.giacone@gmail.com
- BUSNADIEGO, Martin	53020	martinbusnaisi@gmail.com

Introducción Teórica.....	1
Codificación del Problema: Parámetros, Genes y Cromosomas.....	1
Función Objetivo vs. Función de Fitness.....	1
Algoritmo Canónico.....	1
Selección.....	2
Selección por Ruleta.....	2
Selección por Torneo.....	3
Elitismo.....	3
Crossover.....	4
Mutación.....	4
Descripción del problema y función objetivo.....	4
El Problema Matemático.....	4
Diseño del Cromosoma.....	5
Función de Adaptación (Fitness).....	6
Evaluación Estadística: Desviación Estándar del Fitness.....	6
Parámetros fijos y variables (tabla).....	6
Parámetros Fijos.....	7
Parámetros Variables (Escenarios de Prueba).....	7
Experimentos adicionales.....	7
Codificación de la Solución:.....	8
Arquitectura General - motor_ag.py:.....	8
Codificación y Resultados para Opción A (ruleta):.....	9
Código y Lógica de la Ruleta.....	9
Resultados.....	10
Codificación y Resultados para Opción B (torneo):.....	13
Código y Lógica del Torneo.....	13
Resultados.....	13
Codificación y Resultados para Opción C (elitismo):.....	16
Resultados:.....	17
Experimentos adicionales:.....	18
Gráficas comparativas (máximos, mínimos, promedios).....	19
Desempeño de la Selección por Ruleta (Opción A).....	20
Corrida de 20 Generaciones.....	20
Corrida de 100 Generaciones.....	21
Corrida de 200 Generaciones.....	22
Análisis Comparativo de las Corridas (Ruleta).....	23
Desempeño de la Selección por Torneo (Opción B).....	24
Corrida de 20 Generaciones.....	24
Corrida de 100 Generaciones.....	25
Corrida de 200 Generaciones.....	27
Análisis Comparativo de las Corridas (Torneo).....	28
Desempeño opción C (Elitismo).....	29
Desempeño Ruleta con elitismo.....	29

Prueba 1 Ruleta - 20 Generaciones.....	29
Prueba 2 Ruleta - 100 Generaciones.....	30
Prueba 3 Ruleta - 200 Generaciones.....	31
Desempeño torneo con elitismo.....	32
Prueba 1 Torneo - 20 Generaciones.....	32
Prueba 2 Torneo - 100 Generaciones.....	33
Prueba 3 Torneo - 200 Generaciones.....	33
Desempeño de los experimentos adicionales:.....	35
Gráficos y análisis del experimento 1:.....	35
Análisis por fases del gráfico.....	35
Gráficos y análisis del experimento 2:.....	37
Análisis por fases del gráfico.....	38
Gráficos y análisis del experimento 3:.....	40
Análisis por fases del gráfico.....	40
Tabla resumen de estabilidad y tiempos.....	42
Análisis de tiempos de cómputo.....	42
Análisis de la relación costo-beneficio: ¿conviene correr más generaciones?....	43
Análisis de estabilidad y el efecto del elitismo.....	43
Conclusión comparativa general.....	44
Conclusiones y recomendación final.....	44
Conclusiones generales.....	44
Recomendación final.....	45
Anexos:.....	47
Codigo Fuente en python:.....	47

Introducción Teórica

Los Algoritmos Genéticos (AG) son métodos adaptativos empleados para resolver problemas complejos de búsqueda y optimización. Su funcionamiento se inspira en la evolución biológica y en los principios de la selección natural y la supervivencia de los más aptos postulados por Darwin. Para operar, el algoritmo no trabaja directamente con las variables del problema, sino que requiere una abstracción y traducción previa de las posibles soluciones a una estructura de datos procesable computacionalmente.

Codificación del Problema: Parámetros, Genes y Cromosomas

Para que el algoritmo pueda iterar y evolucionar las soluciones, es indispensable establecer una correspondencia entre el modelo matemático del problema y la estructura de datos interna del programa. Esta traducción se realiza mediante tres conceptos fundamentales:

- **Parámetro:** Es la variable real o incógnita del problema matemático que se desea optimizar (por ejemplo, la variable x de una función).
- **Gen:** Es la representación codificada de un parámetro individual. Para poder trabajar con estos valores en la computadora, el parámetro se traduce a un alfabeto informático, generalmente una cadena binaria (ceros y unos). El nivel de precisión requerido o el dominio de la variable determinará la cantidad de bits asignados a este gen.
- **Cromosoma:** Es la estructura de datos completa que agrupa a los genes. Consiste en una tira de símbolos que representa una "solución factible" entera al problema. En problemas de una sola variable, el cromosoma puede estar compuesto por un único gen, pero en problemas multivariantes, el cromosoma es la concatenación de todos los genes. Al conjunto de múltiples cromosomas operando simultáneamente se lo denomina población.

Función Objetivo vs. Función de Fitness

Una vez que las soluciones están codificadas, el algoritmo necesita evaluar su calidad. Es vital distinguir entre dos funciones que actúan en este proceso:

- **Función Objetivo:** Es la fórmula matemática pura del problema real sobre la cual se busca hallar un máximo o un mínimo por ejemplo.
- **Función de Adaptación (Fitness):** Es la métrica interna del algoritmo que evalúa el potencial reproductivo de un cromosoma. Le asigna una puntuación o probabilidad a cada individuo en función de qué tan cerca está de ser la mejor solución. Aunque en muchos casos la función fitness es exactamente igual a la función objetivo, en otros problemas puede definirse de manera distinta (por ejemplo, penalizando soluciones que violan restricciones o normalizando valores).

Algoritmo Canónico

El Algoritmo Genético Canónico (o simple) parte de una población inicial de cromosomas generada completamente al azar. A cada individuo se le computa su función de evaluación

y, para evolucionar hacia poblaciones con mejores soluciones, el algoritmo aplica cíclicamente tres operadores matemáticos fundamentales:

1. **Selección:** Consiste en extraer aleatoriamente pares de cromosomas de la población actual para que actúen como progenitores de la siguiente generación. La regla base es que los individuos con mejor fitness tienen mayor probabilidad de ser seleccionados. Para llevar a cabo este proceso existen distintas estrategias probabilísticas, como el método de la Ruleta (característico del modelo canónico) y el método de Torneo.
2. **Crossover (Cruzamiento):** Es el principal operador de búsqueda. Toma a los pares de padres seleccionados y, condicionado por una probabilidad de cruce predefinida, corta sus cromosomas en un punto aleatorio para intercambiar su material genético. El resultado es la generación de dos descendientes nuevos que combinan los "bloques de construcción" de sus padres.
3. **Mutación:** Actúa como un operador secundario diseñado para mantener la diversidad genética. Con una probabilidad extremadamente baja (usualmente menor al 5%), altera el valor de algún gen de un descendiente (por ejemplo, cambiando un 0 por un 1 en una codificación binaria). Esto evita el estancamiento del algoritmo y asegura que la población no converja de manera prematura hacia un óptimo local.

Selección

Selección por Ruleta

La selección por ruleta es una técnica en la cual la probabilidad de elegir a un individuo para la reproducción es directamente proporcional a su valor de adaptación (fitness).

Matemáticamente, la probabilidad de selección p_i de un individuo se calcula como el cociente entre su fitness individual y la suma total del fitness de la población actual:

$$p_{i,prop} = g(l_i^j) / \sum_{j=1}^n g(l_i^j) \quad (1)$$

En este esquema, se crea una ruleta imaginaria (o pool genético) donde cada individuo ocupa una porción o arco de circunferencia equivalente a su probabilidad p_i . Cuanto mejor sea la adaptación de la solución, mayor será su porción en la ruleta y, por ende, mayor su probabilidad de ser seleccionada múltiples veces. La selección se concreta "girando" la ruleta virtual mediante la generación de un número aleatorio, repitiendo el proceso hasta formar las parejas necesarias para llenar la nueva generación. Este método permite que los individuos superiores dominen la descendencia, pero mantiene una probabilidad no nula de que los peores individuos aporten material genético, preservando así cierta diversidad.

Sin embargo, un problema habitual y documentado de este esquema de selección estático es su susceptibilidad a la pérdida de diversidad. Si en las primeras generaciones (donde la población suele ser pobre) surge un "superindividuo" con un fitness excepcionalmente alto respecto al resto, este acaparará la mayor parte de la ruleta. Al ser seleccionado repetidas veces, la población siguiente se llenará rápidamente de copias casi exactas de este progenitor, destruyendo la diversidad genética necesaria para la exploración y estancando la búsqueda del algoritmo en un óptimo local.

Para mitigar esta vulnerabilidad originada por los “superindividuos”, la teoría evolutiva desarrolló técnicas alternativas que evitan la selección proporcional directa. Una de las soluciones más documentadas es la Selección por Rango (Ranking), un método independiente en el cual se ordena a la población de peor a mejor y se asignan las probabilidades de selección basándose estrictamente en esa posición ordinal, ignorando la magnitud del fitness absoluto. Si bien el rango logra aplanar las diferencias matemáticas y asegurar una repartición más uniforme, otra de las estrategias más eficientes y utilizadas para resolver este mismo problema y controlar la presión selectiva es la Selección por Torneo, la cual se abordará como método alternativo en el presente trabajo.

Selección por Torneo

La selección por torneo se utiliza para elegir los individuos que participarán en la reproducción basándose en competencias directas entre subconjuntos de la población actual.

Se escoge aleatoriamente, con o sin reemplazamiento, un grupo reducido de individuos de la población para conformar el torneo, y se compara su fitness. El individuo con mejor aptitud dentro de ese grupo resulta ganador y es seleccionado como padre para reproducirse. Este proceso se repite tantas veces como sea necesario hasta completar la cantidad de individuos de la nueva población. El parámetro clave de este método es el tamaño del torneo (T); un valor pequeño favorece la diversidad genética al introducir mayor aleatoriedad, mientras que un valor mayor incrementa la presión selectiva, acelerando la convergencia pero con riesgo de perder diversidad de exploración.

En cuanto a las variantes de este método, la literatura sugiere que para torneos pequeños (habitualmente de 2 competidores) suele implementarse una versión probabilística, permitiendo la selección ocasional de individuos subóptimos para preservar la diversidad. Sin embargo, con el objetivo de elevar la presión selectiva en el presente modelo, se optó por implementar un torneo determinístico con un tamaño $T=4$ (equivalente al 40% de la población inicial). En esta configuración, de los competidores seleccionados aleatoriamente, el individuo con mayor fitness resulta invariablemente ganador, acelerando así la convergencia del algoritmo genético.

Elitismo

El Elitismo es una estrategia de optimización que garantiza que las mejores soluciones encontradas hasta el momento se conserven y pasen directamente a la siguiente generación sin sufrir alteraciones por los operadores de cruce o mutación.

En un algoritmo genético puramente canónico existe el riesgo de que, tras aplicar los operadores probabilísticos, el cromosoma con la mejor adaptación se destruya o se pierda. Para evitar esta pérdida y asegurar que el proceso evolutivo sea progresivo, el método de selección elitista interviene copiando directamente a una porción mínima de la población (la “élite”, conformada por el o los individuos con mejor fitness) hacia la nueva población. Desde el punto de vista matemático, el elitismo es la condición que permite garantizar la convergencia global del algoritmo genético; es decir, asegura que al aumentar las iteraciones, la probabilidad de encontrar el óptimo tienda a uno.

En el marco de este trabajo práctico, se implementará una variante conocida teóricamente como elitismo parcial. Esto significa que, en una población de tamaño n , se mantendrá una copia de los m mejores individuos (en este caso, los 2 mejores). Esta élite se transfiere intacta, mientras que los lugares restantes de la nueva población se completan de manera tradicional, muestreando a los individuos restantes de la población inicial para someterlos a los operadores de selección, cruce y mutación correspondientes.

Crossover

El cruce es el operador principal de búsqueda en los algoritmos genéticos. Su función es emular la reproducción sexual, intercambiando material genético entre dos soluciones seleccionadas como "padres" para generar descendencia que combine las mejores características de ambos.

En la arquitectura base de este trabajo práctico se implementa el Cruce basado en 1 punto (el método utilizado por el Algoritmo Genético Canónico). El procedimiento consiste en seleccionar aleatoriamente un punto de corte dentro de la longitud de los cromosomas de ambos padres; a continuación, se intercambian las secciones de la cadena binaria que se encuentran a la derecha de dicho punto de corte para formar dos nuevos individuos.

Mutación

La mutación es considerada un operador secundario que se aplica sobre los individuos generados tras el proceso de recombinación. Su función principal es introducir un pequeño elemento de aleatoriedad para preservar la diversidad genética de la población, evitando así la convergencia prematura hacia óptimos locales y permitiendo explorar nuevas áreas del espacio de búsqueda.

En codificaciones binarias, como la implementada en el presente modelo, el operador clásico (Binary Mutation) consiste en invertir el valor del gen seleccionado (cambiando un 0 por un 1, o viceversa). Siguiendo las recomendaciones teóricas, la probabilidad de mutación (p_m) se mantiene deliberadamente baja, fijándose en un 5%. Un porcentaje excesivamente alto degradaría el algoritmo genético transformándolo en una búsqueda puramente aleatoria, mientras que la ausencia del operador impediría recuperar material genético vital perdido durante las fases de selección y cruce.

Descripción del problema y función objetivo

El objetivo de este trabajo práctico es implementar un Algoritmo Genético Canónico para hallar el valor que maximiza una función matemática unidimensional específica. A continuación, se detalla la naturaleza del problema y cómo sus variables se traducen a la estructura del algoritmo.

El Problema Matemático

El enunciado plantea la búsqueda del máximo global de la siguiente función objetivo:

$$f(x) = (x / \text{coef})^2 \quad (2.1)$$

- **Coeficiente (coef)** = $2^{30} - 1$
- **Dominio de la función:** $x \in [0, 2^{30} - 1]$

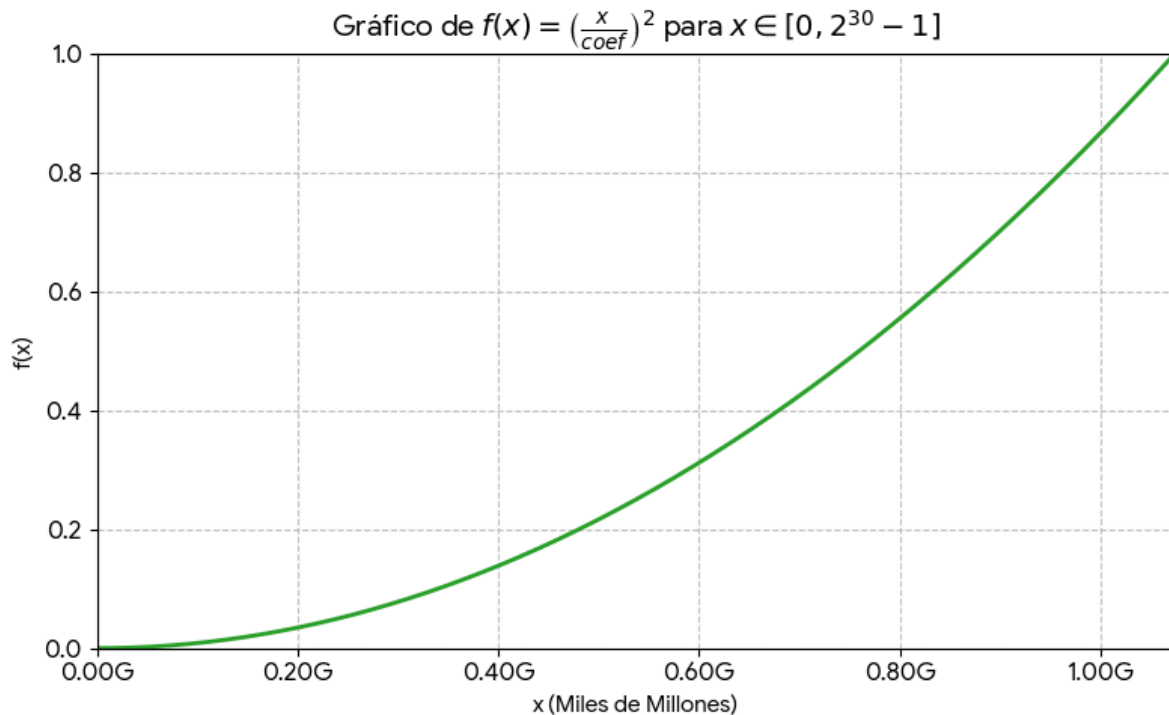


Figura 1: Función Objetivo Normalizada y Dominio del Problema

Al analizar el comportamiento de esta parábola dentro del dominio establecido, se observa que la función está diseñada para estar "normalizada". Dado que el valor máximo que puede tomar la variable x es exactamente igual al denominador (coef), el resultado máximo que devolverá la función objetivo tenderá siempre a 1 (cuando $x = 2^{30} - 1$). Por el contrario, su valor mínimo será 0 (cuando $x = 0$). Esta normalización entre 0 y 1 resulta ideal para la evaluación probabilística que requiere un algoritmo genético.

Diseño del Cromosoma

Para que el algoritmo genético pueda operar sobre este problema matemático, es necesario establecer la codificación de las posibles soluciones. Siguiendo la arquitectura teórica, se definen los siguientes elementos:

1. **El Parámetro:** Al tratarse de un problema unidimensional, existe una única variable independiente a optimizar: el parámetro x .
2. **El Gen y el Cromosoma:** Puesto que solo hay un parámetro en el problema, la estructura de datos que representa una solución (el cromosoma) estará compuesta por un único gen que codificará el valor de x .
3. **Longitud y Alfabeto:** La decisión arquitectónica más importante radica en la longitud de la cadena binaria. El dominio del problema exige que x pueda tomar cualquier valor entero entre 0 y $2^{30} - 1$. En el sistema de numeración binario, el

número decimal $2^{30} - 1$ equivale exactamente al valor máximo representable con 30 bits (una cadena compuesta por treinta 1s consecutivos). Por lo tanto, para poder barrer todo el espacio de soluciones o dominio sin perder información ni desperdiciar memoria, cada cromosoma deberá codificarse estrictamente como un vector binario de 30 posiciones (bits).

Función de Adaptación (Fitness)

En este caso de estudio, no es necesario aplicar penalizaciones ni transformaciones adicionales a la solución. La función de fitness utilizada para evaluar el grado de adaptación de cada cromosoma coincidirá de manera directa con la función objetivo $f(x)$. Al decodificar los 30 bits del cromosoma a su valor entero (obteniendo el parámetro x) y pasarlo por la función, el resultado (entre 0 y 1) dictará la probabilidad directa de supervivencia de dicho individuo en la fase de selección.

Evaluación Estadística: Desviación Estándar del Fitness

Si bien la función de fitness determina la calidad individual de una solución, es necesario cuantificar la dispersión de estas aptitudes a lo largo de toda la población mediante la desviación estándar (σ). Esta métrica actúa como un "termómetro" de la diversidad genética y se calcula para cada generación según la siguiente expresión:

$$\sigma = \sqrt{(\sum_{i=1}^N (f_i - \bar{f})^2) / N} \quad (2.2)$$

Donde $\bar{f}$ representa el fitness promedio de la generación actual.

Interpretación técnica en Algoritmos Genéticos:

El seguimiento de este valor a través de las iteraciones permite diagnosticar el estado del algoritmo sin necesidad de observar cada cromosoma individualmente:

- **Valores altos de σ :** Indican que existe una gran variedad de soluciones en la población. Es una señal positiva en las primeras generaciones, ya que el algoritmo está explorando distintas áreas del espacio de búsqueda y evitando la convergencia prematura.
- **Valores decrecientes de σ :** Es el comportamiento esperado a medida que avanza el algoritmo. Significa que los individuos están comenzando a parecerse entre sí y a agruparse en torno a las mejores soluciones encontradas.
- **Valores cercanos a 0:** Si la desviación estándar llega a cero, significa que toda la población ha convergido hacia el mismo cromosoma (todos los individuos son clones). Esto puede ser interpretado como una convergencia exitosa hacia el óptimo, o bien como un estancamiento en un óptimo local, impidiendo que el algoritmo siga evolucionando debido a la falta de material genético nuevo.

Parámetros fijos y variables (tabla)

Para la ejecución de los distintos escenarios propuestos en este trabajo práctico (Opciones A, B y C), se ha definido una configuración base para el algoritmo genético. A continuación,

se detallan los parámetros que se mantienen constantes en las corridas principales y aquellos que se modifican para evaluar el comportamiento del sistema.

Parámetros Fijos

Estos parámetros se mantienen inalterables durante las pruebas principales para garantizar que las comparaciones entre los distintos métodos de selección sean consistentes.

Parámetro	Valor Base / Método
Probabilidad de Crossover	0.75
Probabilidad de Mutación	0.05
Población Inicial	10 individuos
Ciclos del programa	20
Método de Selección	Ruleta
Método de Crossover	1 Punto
Método de Mutación	Invertida

Parámetros Variables (Escenarios de Prueba)

Estos son los valores que se irán modificando a lo largo del trabajo práctico para cumplir con los requerimientos de cada inciso y observar la evolución de la convergencia.

Escenario	Método de Selección	Ciclos (Iteraciones)
Opción A	Ruleta	20, 100 y 200
Opción B	Torneo	20, 100 y 200
Opción C	Elitismo (2 mejores)	100
Experimentos Adicionales	Ruleta y Torneo	100

Experimentos adicionales

Cabe destacar que los parámetros definidos como "fijos" en la Tabla 1 establecen únicamente la línea base del modelo canónico para evaluar el comportamiento estándar en las Opciones A, B y C.

Sin embargo, en la fase final de Experimentos Adicionales, esta condición de invariabilidad se romperá intencionalmente. Al alterar drásticamente estos valores estructurales (como, por ejemplo, reducir la probabilidad de crossover a valores cercanos a cero o elevar la mutación), el objetivo será someter al algoritmo a condiciones de estrés. Esto permitirá observar empíricamente y documentar fenómenos teóricos adversos —tales como la pérdida de diversidad genética, la convergencia prematura hacia óptimos locales, o la degradación del algoritmo genético hacia una simple búsqueda aleatoria— demostrando así la sensibilidad del sistema y la importancia de una correcta parametrización inicial.

Codificación de la Solución:

Para la implementación del Algoritmo Genético se adoptó una arquitectura modular, dividiendo el código en cuatro archivos con responsabilidades bien delimitadas. Un archivo central (`motor_ag.py`) concentra toda la lógica compartida del algoritmo, mientras que tres archivos independientes (`opcion_a.py`, `opcion_b.py`, `opcion_c.py`) implementan cada variante de selección, invocando al motor central para las operaciones comunes. Esta separación garantiza que cualquier cambio en la lógica base se propague automáticamente a todas las variantes sin duplicar código. Todo el código fuente se puede encontrar en el [Anexo de Código Fuente](#)

Arquitectura General - `motor_ag.py`:

Este archivo actúa como el núcleo del sistema. Define las constantes globales del problema (longitud del cromosoma, tamaño de la población, coeficiente de la función, probabilidades de crossover y mutación) y expone las siguientes funciones utilizadas por todas las opciones:

funcion_objetivo(valor_decimal): implementa la fórmula $f(x) = (x/\text{coef})^2$ y devuelve el valor de adaptación de un individuo dado su representación decimal.

binario_a_decimal(cromosoma): convierte un vector de 30 bits en su equivalente entero, recorriendo cada posición y acumulando la potencia de 2 correspondiente.

calcular_desviacion_estandar(valores): calcula la desviación estándar poblacional del fitness en cada generación, utilizada como indicador de diversidad genética.

cruce_un_punto(p1, p2): aplica el operador de cruce condicionado a la probabilidad P_c . Si el cruce ocurre, selecciona un punto de corte aleatorio e intercambia los segmentos derechos de ambos padres, devolviendo dos hijos y el punto utilizado.

mutar_individuo(cromosoma): aplica la mutación invertida condicionada a la probabilidad P_m . Si la mutación ocurre, selecciona una posición aleatoria e invierte el bit correspondiente.

crear_poblacion(): genera la población inicial de 10 cromosomas aleatorios de 30 bits cada uno.

obtener_elite(poblacion, fitness): ordena la población por fitness de forma descendente y devuelve los 2 mejores individuos, utilizados cuando el elitismo está activo.

Adicionalmente, el motor contiene tres funciones de salida que centralizan toda la generación de resultados:

graficar_y_guardar(...): produce el gráfico de convergencia con las curvas de máximo, mínimo, promedio y desviación estándar usando matplotlib, e incluye en el título los parámetros de la corrida para trazabilidad completa.

exportar_csv_generaciones(...): vuelca el historial generación por generación a un archivo CSV, registrando para cada iteración el máximo, mínimo, promedio, desviación estándar, el cromosoma del mejor individuo y su valor decimal.

guardar_resumen_global(...): mantiene una tabla CSV consolidada con una fila por configuración ejecutada (método, generaciones, elitismo, mejor fitness, generación en que se alcanzó y tiempo). Utiliza deduplicación automática para que cada combinación de parámetros conserve únicamente el resultado de la última corrida.

generar_reporte_html(...): construye un reporte visual interactivo en HTML que permite navegar generación por generación, mostrando el estado de la población, la distribución de la ruleta o los torneos realizados, los cruces con el punto de corte señalado, las mutaciones destacadas en el cromosoma afectado, y los individuos élite seleccionados cuando corresponde.

A partir de esta base común, cada opción implementa únicamente su operador de selección propio y delega el resto al motor. Las secciones siguientes detallan esa implementación particular para cada variante.

Codificación y Resultados para Opción A (ruleta):

Código y Lógica de la Ruleta

La Opción A implementa el método de selección por ruleta dentro de la función **ejecutar_ruleta(generaciones, elitismo=False)**. Esta función recibe la cantidad de generaciones a ejecutar y un parámetro opcional que habilita el elitismo, lo que permite reutilizarla tanto para la Opción A pura como para la comparativa de la Opción C.

Al inicio de cada generación, se evalúa toda la población con **funcion_objetivo** y **binario_a_decimal**, calculando los valores objetivos y normalizándolos para obtener el fitness proporcional de cada individuo. Con estos valores se identifica el mejor individuo de la generación y sus estadísticas, que se almacenan en el historial mediante **exportar_csv_generaciones**.

El operador de selección se construye a través de **obtener_padre_ruleta(poblacion, ruleta_vector)**, que implementa la ruleta mediante un vector de 100 posiciones precalculado.

Cada individuo ocupa en ese vector una cantidad de casilleros proporcional a su fitness, mapeando la probabilidad continua a un espacio discreto de 100 slots. Un número aleatorio entre 0 y 99 determina qué individuo es seleccionado, replicando el "giro" de la ruleta imaginaria de forma eficiente sin búsquedas costosas. Para garantizar que la suma de casilleros sea exactamente 100, se aplica un ajuste de precisión que suma o resta un casillero al individuo con mayor porción hasta alcanzar el total exacto.

Con los padres seleccionados, se invocan **cruce_un_punto** y **mutar_individuo** del motor central para generar los hijos de cada par. Si el elitismo está activo, los dos mejores individuos obtenidos por **obtener_elite** se transfieren directamente a la nueva población antes de completar el resto por selección. Al finalizar todas las generaciones, se registra el

tiempo de ejecución y se delegan las salidas a **graficar_y_guardar** y **guardar_resumen_global**.

Resultados

A continuación, se presentan las salidas del algoritmo genético canónico (Selección por Ruleta, sin elitismo) variando la cantidad de generaciones. En cada prueba se evaluó la evolución del valor máximo, mínimo y promedio (cuyas gráficas de convergencia se encuentran adjuntas en la Sección 8 del presente informe), junto con la desviación estándar del fitness.

Prueba 1: 20 Generaciones (Extracto del archivo de ejecución)

Gen	Max	Min	Prom	Desv_Fit
1	0.868226	0.000786	0.464921	0.077455
2	0.876658	0.601457	0.744849	0.011732
3	0.877891	0.654959	0.763964	0.011039
...	...	...	...	...
18	0.861499	0.854263	0.860413	0.000269
19	0.861499	0.857877	0.860774	0.000168
20	0.861499	0.857877	0.860412	0.000192

Mejor individuo encontrado: El cromosoma correspondiente al valor máximo histórico en esta corrida fue 11101111101110010000010011111, cuyo valor decimal decodificado (1006051487) se traduce en un objetivo máximo de **0.877891** (alcanzado tempranamente en la Generación 3).

Análisis de Convergencia: Al observar la desviación estándar del fitness, se nota que la corrida inicia con una alta diversidad poblacional ($\sigma \approx 0.077$). Al tratarse de tan solo 20 iteraciones, el algoritmo se mantiene predominantemente en una fase de exploración del espacio de soluciones. Aunque el promedio sube y la desviación baja levemente, la convergencia está lejos de lograrse, lo que justifica por qué el "mejor individuo" aparece en la generación 3 pero luego los valores fluctúan.

Prueba 2: 100 Generaciones (Extracto del archivo de ejecución)

Gen	Max	Min	Prom	Desv_Fit
1	0.950240	0.010695	0.442238	0.079615
2	0.950240	0.068539	0.691727	0.040954
3	0.950240	0.642153	0.838413	0.015508
...	...	...	...	...
98	0.966042	0.834552	0.941799	0.003821
99	0.967959	0.834552	0.931520	0.005242
100	0.966042	0.834552	0.931328	0.005228

Mejor individuo encontrado:: El cromosoma correspondiente al valor máximo histórico en esta corrida fue 111111011000110100100001001111, alcanzando un objetivo máximo de **0.980960** (equivalente al decimal 1063471183). Este individuo apareció en la Generación 21, aunque luego se perdió por el efecto natural de cruce y selección sin elitismo.

Análisis de Convergencia: En este escenario, la desviación estándar muestra una caída pronunciada y constante, pasando de una alta dispersión inicial a valores muy bajos ($\sigma \approx 0.005$) en las últimas iteraciones. Esto indica visual y numéricamente que la población está perdiendo diversidad y comenzando a converger. Un fenómeno vital observado aquí es el impacto de trabajar sin elitismo: el individuo óptimo absoluto se encontró en la Generación

21, pero al no estar protegido, el cruce y la mutación lo terminaron destruyendo, haciendo que las últimas generaciones se estabilicen en un máximo local inferior.

Prueba 3: 200 Generaciones (Extracto del archivo de ejecución)

Gen	Max	Min	Prom	Desv_Fit
1	0.886241	0.001543	0.198086	0.127466
2	0.886241	0.169145	0.510048	0.060537
3	0.886243	0.169143	0.632078	0.049475
...	...	...	...	...
198	0.992195	0.992188	0.992194	0.000000
199	0.992195	0.871590	0.980134	0.003691
200	0.992195	0.992195	0.992195	0.000000

Mejor individuo encontrado: El cromosoma correspondiente al valor máximo definitivo de la prueba fue 111111101111111111000010011111, con un valor objetivo de **0.992195** (equivalente al decimal 1069543583, descubierto en la Generación 188 y mantenido hacia el final).

Análisis de Convergencia: La tabla de 200 iteraciones expone el comportamiento asintótico y la convergencia absoluta del algoritmo. Al observar las últimas filas (generaciones 198 a 200), los valores de Máximo, Mínimo y Promedio se igualan por completo en 0.992195. En consecuencia, la desviación estándar del fitness cae exactamente a 0. Esto demuestra que la población perdió toda su diversidad genética (los 10 individuos son clones exactos del mejor), estancando el proceso evolutivo ya que el operador de cruce deja de tener efecto útil al recombinar padres idénticos.

Codificación y Resultados para Opción B (torneo):

Código y Lógica del Torneo

La Opción B implementa la selección por torneo dentro de **ejecutar_torneo(generaciones, elitismo=False)**, con una estructura idéntica a la de la Opción A en cuanto a evaluación, registro de estadísticas y delegación de salidas al motor central. La diferencia sustancial reside en el operador de selección, encapsulado en la función **seleccion_torneo(poblacion, fitness)**.

Esta función define internamente **elegir_padre()**, que selecciona aleatoriamente k individuos distintos de la población, donde k se calcula como el 40% del tamaño poblacional (en este caso $k=4$). Entre los k candidatos, el ganador es siempre el de mayor fitness, implementando un torneo determinístico sin componente probabilística. La función se llama dos veces por par reproductivo para obtener ambos padres, y retorna además los índices de los aspirantes para su visualización en el reporte HTML.

A diferencia de la ruleta, este mecanismo no requiere construir ni mantener ningún vector de probabilidades acumuladas: la competencia directa entre subconjuntos de la población reemplaza la normalización global, haciendo al torneo menos sensible a la presencia de individuos con fitness excepcionalmente alto que distorsionarían la ruleta. El resto del ciclo, crossover, mutación, incorporación de élite si corresponde y actualización de la población, es exactamente igual al de la Opción A, ejecutándose sobre la misma infraestructura del motor central.

Resultados

Prueba Torneo 1: 20 Generaciones

Gen	Max	Min	Prom	Desv_Fit
1	0.967921	0.000295	0.259379	0.133061
2	0.968019	0.140035	0.491662	0.076640
3	0.968030	0.905200	0.955434	0.002629

...	...	...	...	...
18	0.999020	0.999020	0.999021	0.000000
19	0.999020	0.999020	0.999021	0.000000
20	0.999021	0.998777	0.998996	0.000007

Mejor individuo encontrado: El cromosoma correspondiente al valor máximo histórico de la prueba fue 11111111101111111100111100001 (decimal: 1073215969), con un valor objetivo de **0.999021** descubierto en la Generación 18 y mantenido hacia el final.

Análisis de Convergencia: La tabla de 20 iteraciones muestra la convergencia extremadamente rápida característica del torneo. Observando las últimas filas (generaciones 18-20), los valores de Máximo permanecen constantes en 0.999021, aunque el Mínimo y Promedio presentan pequeñas fluctuaciones (0.999021 a 0.998777). La desviación estándar del fitness cae desde 0.133 (Gen 1) a casi 0.000000 en la Gen 7, demostrando que la población alcanzó homogenización completa en solo 7 generaciones. A partir de este punto, las pequeñas variaciones observadas en Gen 20 (Desv_Fit = 0.000007) son consecuencia de mutaciones puntuales que no logran generar individuos significativamente mejores. El torneo sin elitismo exhibe aquí su principal debilidad: aunque encuentra un máximo local excelente (0.999021) muy rápidamente, pierde diversidad genética de forma casi instantánea, impidiendo que futuras generaciones exploren mejoras.

Prueba Torneo 2: 100 Generaciones

Gen	Max	Min	Prom	Desv_Fit
1	0.921971	0.005710	0.316252	0.087466
2	0.921971	0.335927	0.568994	0.032402
3	0.957800	0.504184	0.766421	0.025851

...	...	...	...	...
98	0.992187	0.989214	0.992187	0.000000
99	0.992187	0.989214	0.992187	0.000000
100	0.992187	0.989214	0.992187	0.000000

Mejor individuo encontrado: El cromosoma correspondiente al valor máximo histórico de la prueba fue 11111101101111001011011110001 (decimal: 1068996337), con un valor objetivo de **0.992187** descubierto alrededor de la Generación 28 y mantenido hacia el final.

Análisis de Convergencia: Con 100 generaciones, el torneo evidencia un comportamiento dual de búsqueda. En las primeras 28 generaciones, el máximo mejora de 0.921971 a 0.992187, fase durante la cual la desviación estándar fluctúa significativamente (desde 0.087466 hasta valores cercanos a 0). A partir de la Generación 28, la población se estabiliza completamente: observando las últimas filas (generaciones 98-100), los valores de Máximo, Mínimo y Promedio permanecen exactamente en 0.992187, y la desviación estándar del fitness es 0.000000. Esto demuestra que la población se transformó en un clon perfecto del mejor individuo encontrado. Sin elitismo y con diversidad genética nula desde la Gen 28, el algoritmo no puede escapar de este máximo local para buscar soluciones potencialmente mejores. La estabilización temprana a los 0.992187 indica que el torneo alcanzó un óptimo local sólido pero quedó atrapado en él, sin exploración adicional.

Prueba Torneo 3: 200 Generaciones

Gen	Max	Min	Prom	Desv_Fit
1	0.965399	0.001783	0.333661	0.097798
2	0.965399	0.439302	0.741901	0.027534
3	0.965399	0.751687	0.922626	0.009264
...	...	...	...	...

198	1.000000	1.000000	1.000000	0.000000
199	1.000000	1.000000	1.000000	0.000000
200	1.000000	1.000000	1.000000	0.000000

Mejor individuo encontrado: El cromosoma correspondiente al valor máximo definitivo de la prueba fue 11111111111111111111111111111111 (todos los bits en 1, decimal: 1073741823), alcanzando un valor objetivo perfecto de **1.000000** descubierto en la Generación 130 y mantenido hacia el final.

Análisis de Convergencia: La tabla de 200 iteraciones expone el comportamiento de búsqueda a largo plazo del torneo. Durante las primeras 128 generaciones, la población mejora gradualmente (0.965399 → 0.968994), alternando entre períodos de homogenización completa ($\sigma = 0$) y pequeños puntos de ruptura causados por mutación. En la Generación 129-130 ocurre un breakthrough evolutivo: se descubre el cromosoma óptimo absoluto con todos sus bits en 1, alcanzando el objetivo máximo posible de 1.000000. Este hallazgo tardío (Gen 130) demuestra que aunque el torneo converge muy rápido a máximos locales, el proceso de exploración mediante mutación eventualmente lo conduce al óptimo global. Observando las últimas filas (generaciones 198-200), los valores de Máximo, Mínimo y Promedio se igualan por completo en 1.000000. En consecuencia, la desviación estándar del fitness cae exactamente a 0.000000. Esto demuestra que la población perdió toda su diversidad genética: los 10 individuos son clones exactos del óptimo global. El operador de cruce deja de tener efecto útil al recombinar padres idénticos, pero en este caso el estancamiento es deseado porque ya se alcanzó la mejor solución posible en el espacio de búsqueda.

Codificación y Resultados para Opción C (elitismo):

Código y lógica de Elitismo:

La Opción C no introduce un nuevo archivo de lógica evolutiva sino que actúa como un orquestador. El archivo `opcion_c.py` define `ejecutar_comparativa_elitismo()`, que invoca secuencialmente `ejecutar_ruleta(100, elitismo=True)` de la Opción A y `ejecutar_torneo(100, elitismo=True)` de la Opción B, activando el parámetro de elitismo en ambos casos.

El mecanismo de elitismo en sí está implementado en el motor central a través de `obtener_elite(poblacion, fitness)`. Esta función combina la población con su vector de fitness, ordena el resultado de forma descendente y extrae los cromosomas de los primeros individuos, con un mínimo de 2 y un máximo del 20% de la población. Los cromosomas extraídos se copian de forma independiente para evitar que modificaciones posteriores por cruce o mutación afecten retroactivamente a los individuos guardados.

En el ciclo de cada generación, la élite se obtiene antes de construir la nueva población. Los individuos élite se insertan directamente al comienzo de la lista de la nueva generación, ocupando sus lugares de forma garantizada. El resto de los lugares se completa mediante el operador de selección correspondiente (ruleta o torneo según la variante) hasta alcanzar el tamaño poblacional completo. De esta forma, el elitismo no reemplaza al mecanismo de selección sino que lo complementa, reservando una porción fija de la nueva generación para los mejores individuos mientras el resto sigue siendo producto de la evolución estocástica normal.

Resultados:

A partir de los resultados obtenidos en las distintas corridas se puede observar con claridad el impacto que tiene el uso del elitismo sobre el comportamiento del algoritmo genético. En términos generales, el elitismo favorece una convergencia más rápida hacia la función objetivo al preservar los mejores individuos de cada generación, impidiendo que soluciones de alta calidad se pierdan por efecto del cruce o la mutación. Esto se refleja directamente en los valores máximos alcanzados, donde el método con elitismo consistentemente termina más cercano a 1 que su contraparte sin importar la cantidad de generaciones ejecutadas.

Sin embargo, esta ventaja tiene un costo: el elitismo reduce significativamente la diversidad de la población. Al proteger al 20% superior, los cromosomas tienden a homogeneizarse rápidamente, lo que se evidencia en los desvíos estándar cercanos a cero que se sostienen durante largas rachas de generaciones. En este escenario los cruces dejan de aportar variación real, ya que los cromosomas son prácticamente iguales entre sí, y el único mecanismo que permite seguir avanzando hacia el óptimo es la ocurrencia de una mutación favorable.

El método sin elitismo, por su parte, mantiene una mayor diversidad a lo largo de toda la corrida. Su desvío estándar presenta picos y recuperaciones cíclicas que demuestran que la población nunca pierde completamente su capacidad de explorar el espacio de soluciones. Si bien sus valores máximos finales son generalmente inferiores a los del método con elitismo, su promedio se mantiene estable y elevado durante más generaciones, lo que indica un equilibrio más saludable entre exploración y explotación.

Comparación desvío fitness:

Ruleta 20 generaciones	Sin Elitismo	Con Elitismo
	Desv_fit	
1	0.0774549867846767	0.08723442973933944

2	0.011732815600168766	0.04349797588984559
3	0.011039446435149123	0.02456480870432923
...	...	...
18	0.0002692996081062324	8.221651427961723e-10
19	0.00016830631998864372	2.2416423212304553e-09
20	0.00019290046227417643	2.410022880234746e-09

Estos resultados se replican en todos los experimentos explicados por la naturaleza propia del elitismo conservando siempre los mejores genes, si no hay un gen mejor este perdurará.

Experimentos adicionales:

El rendimiento de un Algoritmo Genético depende de un delicado equilibrio entre la **explotación** (combinar soluciones buenas, guiado por el Crossover) y la **exploración** (descubrir nuevas soluciones, guiado por la Mutación).

Para comprobar empíricamente cómo reacciona nuestro modelo ante el desbalance de estas fuerzas, se plantearon tres experimentos aislando variables. Todos se ejecutarán sobre 100 iteraciones sin elitismo, partiendo de los valores base dados por la catedra ($P_c = 0.75$ y $P_m = 0.05$):

- **Experimento 1: Pérdida de Explotación (Torneo con $P_c = 0.05$)**

Modificación: Se reduce el crossover drásticamente al 5% utilizando el método de Torneo.

Expectativa: Al tener una alta presión selectiva (Torneo) pero casi nula recombinación de genes, esperamos que los individuos simplemente se clonen. La hipótesis es que observaremos una convergencia prematura casi inmediata y un estancamiento en un óptimo local bajo.

- **Experimento 2: Pérdida de Exploración (Ruleta con $P_m = 0.00$)**

Modificación: Se anula por completo la mutación (0%) utilizando el método de Ruleta.

Expectativa: Sin inyección de nuevo material genético, el algoritmo solo cuenta con los genes de la población inicial (Generación 0). Esperamos que, tarde o temprano, la diversidad se agote y la población se estanque sin alcanzar el óptimo global.

- **Experimento 3: Ruido Extremo (Ruleta con $P_m = 0.40$)**

Modificación: Se eleva la mutación a un valor atípicamente alto (40%).

Expectativa: Esperamos que el algoritmo pierda su capacidad heurística y degele en una búsqueda puramente aleatoria. Gráficamente, se espera observar una alta inestabilidad, ya que la mutación masiva destruirá las buenas soluciones antes de que el crossover pueda aprovecharlas.

Gráficas comparativas (máximos, mínimos, promedios)

En esta sección se exponen visualmente las curvas de convergencia poblacional. El análisis gráfico permite observar de manera directa el comportamiento de los valores Máximos, Promedios y Mínimos de la función objetivo, así como la fluctuación de la desviación estándar (línea roja), la cual funciona como indicador principal de la pérdida de diversidad genética.

Desempeño de la Selección por Ruleta (Opción A)

Corrida de 20 Generaciones

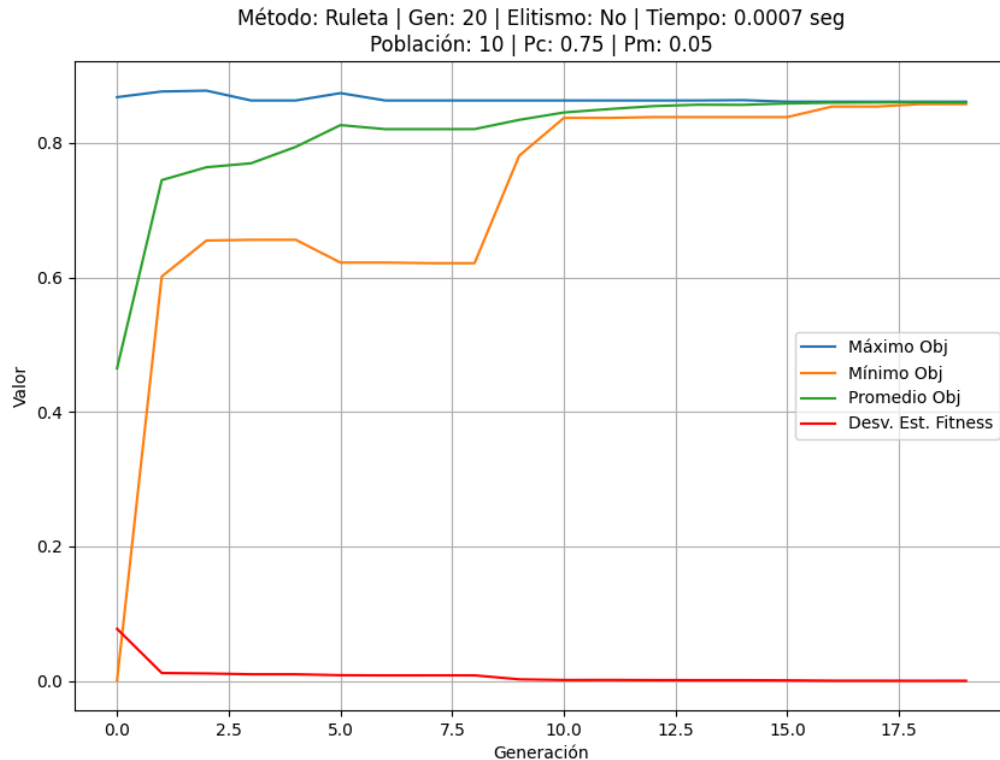


Figura 2: Curvas de convergencia poblacional utilizando Selección por Ruleta sin elitismo (20 Generaciones).

Observaciones:

Desviación Estándar (Línea Roja): Inicia con un pico de dispersión alto (0.077 en la generación 1), lo que indica una población aleatoria y diversa. Cae abruptamente en la segunda generación y luego se estabiliza cerca del eje horizontal (0.0001), evidenciando una rápida pérdida de diversidad inicial, aunque sin llegar al cero absoluto.

Valor Máximo (Línea Azul): Alcanza su pico de manera temprana en la generación 3 (0.877). Sin embargo, luego retrocede y se estanca en 0.861 hacia el final. Esto ilustra cómo la falta de elitismo permite que el cruce degrade la mejor solución.

Valor Promedio (Línea Verde): Acompaña la mejora poblacional subiendo de 0.464 a 0.860, demostrando que, en términos generales, la población evolucionó favorablemente respecto a su estado inicial.

Valor Mínimo (Línea Naranja): Presenta una inestabilidad extrema (arranca casi en 0 y salta constantemente). Representa a los peores individuos generados, producto del operador de mutación actuando sobre una población aún no convergente.

Corrida de 100 Generaciones

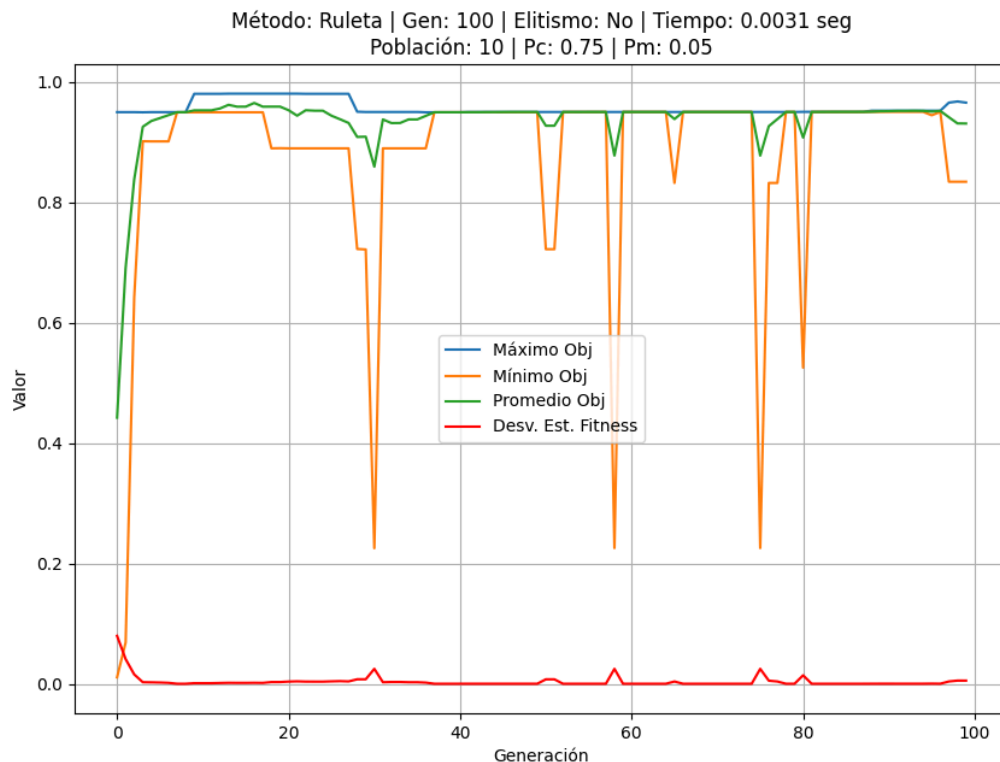


Figura 3: Curvas de convergencia poblacional utilizando Selección por Ruleta sin elitismo (100 Generaciones).

Observaciones:

Desviación Estándar (Línea Roja): Muestra una caída drástica durante las primeras 30 generaciones y luego tiende a aplanarse (oscilando en 0.005), confirmando la estabilización genética. Sin embargo, se observan picos aislados de dispersión a lo largo de la corrida (como en las generaciones 35, 51, 59, 75 y 80). Estos picos representan visualmente la acción del operador de mutación (5%), el cual altera aleatoriamente un cromosoma y rompe la homogeneidad poblacional por un instante, inyectando una diversidad que luego es purgada por la selección.

Valor Máximo (Línea Azul): Revela el mayor defecto del método canónico. En la generación 21, logra descubrir un individuo excelente (0.980), pero al no estar protegido, es destruido en iteraciones posteriores, finalizando la corrida en un máximo local inferior de 0.966.

Valor Promedio (Línea Verde): Se acopla a la tendencia de la línea de máximos, pero manteniéndose siempre un escalón por debajo (0.931 final). Un detalle analítico notable es la presencia de caídas abruptas (picos hacia abajo) que coinciden sincrónicamente con las caídas en la curva de mínimos. Esto ocurre porque, al operar con una población de tamaño reducido ($N=10$), el promedio poblacional es altamente sensible a la aparición de valores atípicos. Cuando el operador de mutación degrada severamente a un individuo (provocando el pico mínimo), el resto de la población —que ya se encuentra cerca del límite superior de la función— no tiene margen matemático para "compensar" esa pérdida de fitness, arrastrando inevitablemente la media generacional hacia abajo de forma visible.

Valor Mínimo (Línea Naranja): Presenta fluctuaciones constantes y caídas repentinas, evidenciando que el 5% de mutación sigue inyectando anomalías que son purgadas generación tras generación.

Corrida de 200 Generaciones

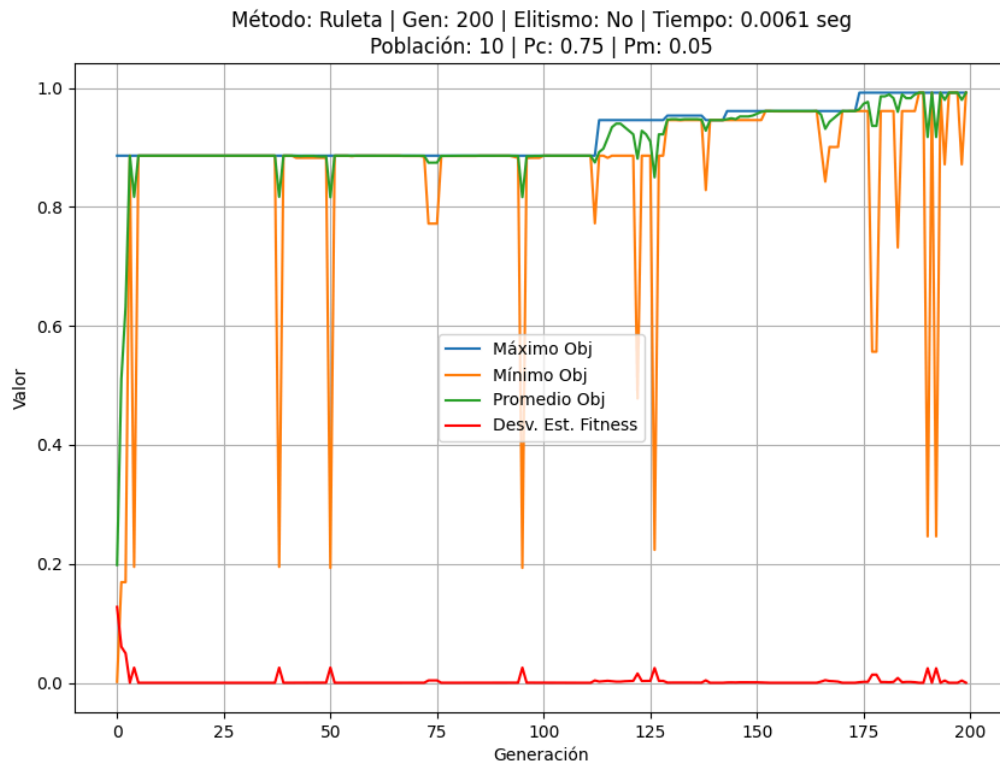


Figura 4: Curvas de convergencia poblacional utilizando Selección por Ruleta sin elitismo (200 Generaciones).

Observaciones:

Desviación Estándar (Línea Roja): Cae progresivamente hasta alcanzar matemáticamente el valor exacto de 0.0000 en las iteraciones finales, dictaminando la saturación total del algoritmo. Previo a esta saturación absoluta, la curva expone múltiples picos de dispersión (ej. iteraciones 31, 50, 94, 180, etc.). Estas "pulsaciones" demuestran el esfuerzo constante del operador de mutación por introducir nuevo material genético en una población que está intentando converger prematuramente, validando que el algoritmo mantiene su capacidad de exploración viva incluso en etapas avanzadas.

Valor Máximo (Línea Azul): Encuentra el valor 0.992195 en la generación 188 y, esta vez, logra mantenerlo hasta la iteración 200 debido a la saturación poblacional extrema.

Valor Promedio (Línea Verde): Se fusiona visual y numéricamente con la línea de máximos en las últimas iteraciones. A nivel algorítmico, que el promedio sea idéntico al máximo absoluto significa que el 100% de la población está compuesta por copias (clones) del mismo individuo. Esta superposición gráfica representa el estado de convergencia definitiva: la ruleta ha seleccionado tantas veces a los cromosomas superiores que se ha perdido todo el material genético alternativo, imposibilitando que el operador de cruce pueda generar nuevas y mejores soluciones.

Valor Mínimo (Línea Naranja): A lo largo de casi toda la ejecución, esta curva presenta una extrema volatilidad con valles muy pronunciados. Estas caídas profundas constantes ilustran la generación de individuos defectuosos debido a la acción del cruce y la mutación sobre genes de alto impacto. Sin embargo, al tener un fitness casi nulo, la ruleta los purga rápidamente en la iteración siguiente (generando el "rebote" inmediato de la curva). Al final de la corrida, la línea se estabiliza y se iguala a las otras dos en 0.992195, con la excepción de una última caída abrupta (0.871 en la gen 199), siendo el rastro final de una mutación inyectando ruido en una población ya clónica.

Análisis Comparativo de las Corridas (Ruleta)

El análisis integral de las gráficas de 20, 100 y 200 iteraciones revela que la Selección por Ruleta es un método efectivo para guiar a la población hacia zonas de alta adaptación, logrando en todos los casos un incremento sostenido del fitness promedio. Sin embargo, sufre de dos vulnerabilidades críticas en su versión canónica. En primer lugar, la falta de una política de **elitismo** provoca "amnesia evolutiva": el algoritmo es capaz de encontrar soluciones casi perfectas, pero las destruye accidentalmente mediante el cruce o la mutación en generaciones posteriores. En segundo lugar, al operar con una población reducida ($N=10$), la ruleta induce una **convergencia prematura y saturación genética** a largo plazo. Al seleccionar iterativamente a los mejores, el pool genético pierde toda su diversidad (evidenciado por la desviación estándar nula en las 200 iteraciones), convirtiendo a la población en clones. En este estado de estancamiento, el operador de cruce queda inutilizado, y las mutaciones esporádicas, en lugar de ayudar a explorar, se convierten en individuos letales que son purgados inmediatamente, impidiendo que el algoritmo escape de los máximos locales.

Desempeño de la Selección por Torneo (Opción B)

Corrida de 20 Generaciones

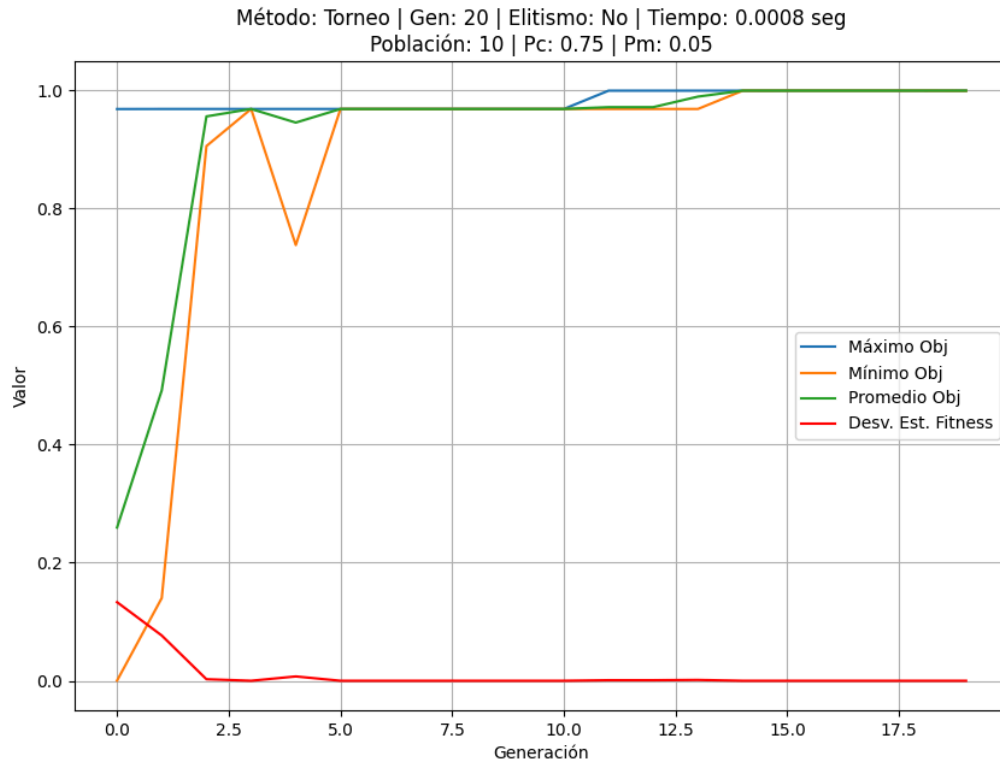


Figura 5: Curvas de convergencia poblacional utilizando Selección por Torneo sin elitismo (20 Generaciones).

Observaciones:

Desviación Estándar (Línea Roja): Inicia con un pico pronunciado de dispersión (0.133 en la generación 1), indicando una población inicial aleatoria y diversa. Presenta un colapso catastrófico casi instantáneo: la línea cae de 0.133 a prácticamente 0.000 en tan solo 7 generaciones, alcanzando valores de homogenización absoluta ($\sigma \approx 0.0000$). Este comportamiento es radicalmente más acelerado que en ruleta, demostrando que la presión selectiva del torneo es mucho más agresiva para la diversidad genética. Una vez alcanzado el valor nulo, la línea permanece pegada al eje horizontal, evidenciando que la población ha perdido todo su material genético alternativo.

Valor Máximo (Línea Azul): Alcanza un valor estable aproximado de 0.999 a partir de la generación 12, manteniéndose en este nivel hasta el final (generación 20) sin fluctuaciones significativas. A diferencia de ruleta (que encontró 0.878 en Gen 3 pero descendió a 0.861 por falta de elitismo), el torneo localiza una solución casi óptima y la preserva, gracias a que toda la población converge a clones del mejor individuo. La línea forma una meseta horizontal desde Gen 12 en adelante.

Valor Promedio (Línea Verde): Acompaña la mejora poblacional subiendo de manera acelerada desde 0.259 (Gen 1) hasta fusionarse visualmente con la línea azul aproximadamente en la generación 7. Esta superposición de las líneas azul y verde a partir de Gen 7 indica que la población está compuesta por clones del mejor individuo. El acoplamiento visual de ambas líneas es la firma de la convergencia prematura pero exitosa característica del torneo.

Valor Mínimo (Línea Naranja): Presenta una inestabilidad inicial extrema (arranca cercano a 0 y salta constantemente entre 0.1 y 0.7 durante Gen 1-5). Luego se estabiliza progresivamente hacia Gen 7 en adelante, aproximándose a los valores de máximo y promedio (~0.968-0.999). Este patrón indica que la población muy rápidamente se homogeniza, dejando poco margen para individuos significativamente inferiores.

Corrida de 100 Generaciones

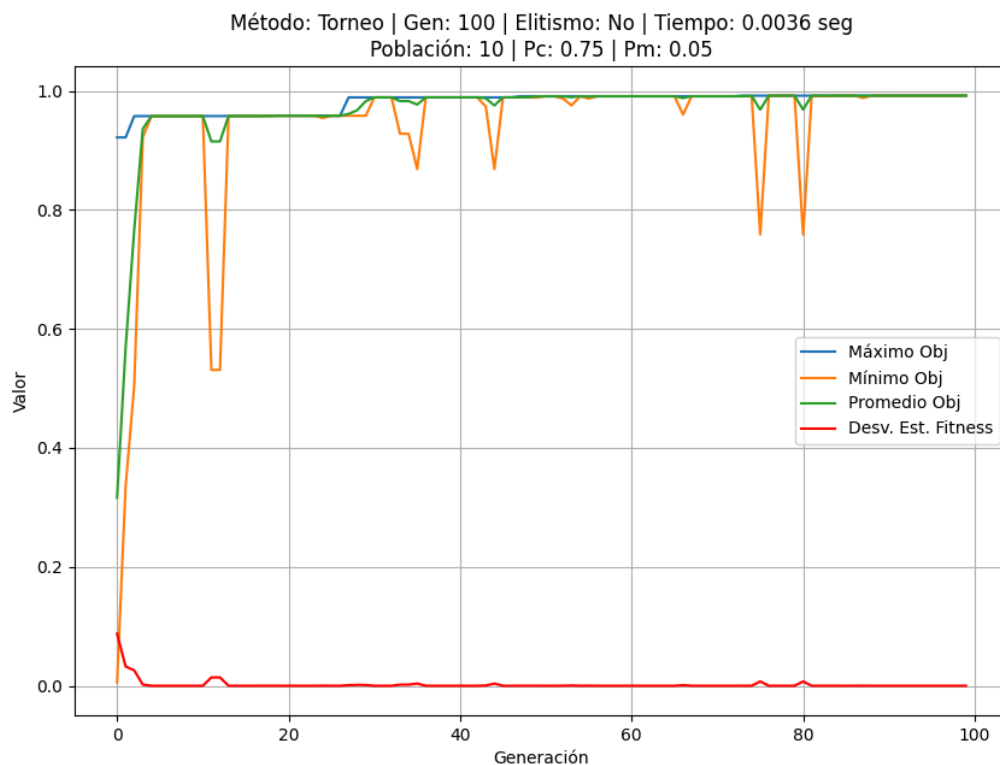


Figura 6: Curvas de convergencia poblacional utilizando Selección por Torneo sin elitismo (100 Generaciones).

Observaciones:

Desviación Estándar (Línea Roja): Similar al patrón de 20 generaciones, cae drásticamente durante las primeras ~20 generaciones (de 0.087 a casi 0). Sin embargo, a diferencia de la corrida corta, se observan múltiples pulsaciones pequeñas pero consistentes dispersas a lo largo de toda la ejecución (visibles alrededor de Gen 10, 18, 28,

35, 50, 60, 70, 80, 90). Estas pulsaciones representan visualmente la acción del operador de mutación (5%), el cual altera aleatoriamente cromosomas en una población ya homogénea, inyectando temporalmente diversidad que luego es purgada por la selección torneo. A diferencia de ruleta (que exhibía picos aislados y altos), torneo genera un patrón de ruido de alta frecuencia y baja amplitud, indicando que el 5% de mutación está constantemente siendo "borrado" por la presión selectiva.

Valor Máximo (Línea Azul): Presenta una forma de "escalera" con dos mesetas bien definidas. Inicia en ~ 0.922 (Gen 1), sube abruptamente a ~ 0.958 en Gen 5 y se estabiliza. Luego, alrededor de la generación 28, identifica una meseta superior en ~ 0.989 , donde permanece constante hasta la generación 100. A diferencia de ruleta 100 gen (que encontró 0.980960 en Gen 21 pero descendió a 0.966042 por pérdida de la solución), el torneo nunca pierde el máximo encontrado, resultando en una solución final superior (0.992187 vs 0.966042). La línea azul es prácticamente una meseta horizontal a partir de Gen 28.

Valor Promedio (Línea Verde): Acompaña estrechamente la línea de máximos, con un desfase mínimo. Desde aproximadamente Gen 28 hacia adelante, ambas líneas se solapan completamente, evidenciando que prácticamente toda la población está compuesta por copias exactas del mejor individuo. Este acoplamiento gráfico es indicativo de la convergencia definitiva del algoritmo a un único fenotipo.

Valor Mínimo (Línea Naranja): Exhibe un comportamiento caótico y errático, con caídas violentas a lo largo de toda la ejecución (formando picos hacia abajo alrededor de Gen 10, 18, 35, 50, 60, 70, 90). Estas caídas profundas coinciden sincrónicamente con los picos observados en la línea roja de desviación, indicando que cuando la mutación degrada severamente a un individuo, inyecta momentáneamente diversidad que es capturada tanto en el gráfico como en la desviación estándar. Sin embargo, estas anomalías son purgadas casi inmediatamente en la siguiente generación por la presión selectiva. El patrón de inestabilidad es más ruidoso en torneo que en ruleta, sugiriendo que el torneo es más sensible a mutaciones en poblaciones clonadas.

Corrida de 200 Generaciones

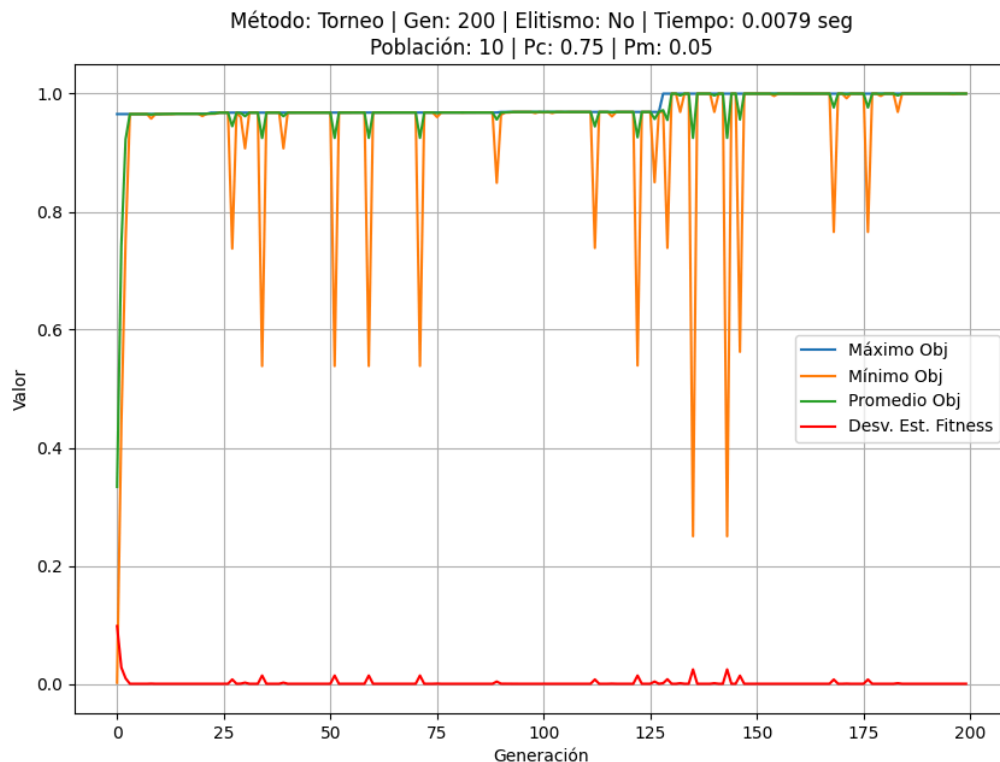


Figura 7: Curvas de convergencia poblacional utilizando Selección por Torneo sin elitismo (200 Generaciones).

Observaciones:

Desviación Estándar (Línea Roja): Cae dramáticamente durante las primeras ~10 generaciones (de 0.098 a casi 0). Luego, exhibe un patrón distintivo: pulsaciones constantes y distribuidas a lo largo de toda la ejecución, visibles desde Gen 10 hasta Gen 200. Estas pulsaciones son de amplitud pequeña pero extremadamente frecuentes, más notables que las observadas en ruleta 200 gen. El fenómeno sugiere que el torneo mantiene una "respiración genética" continua: el 5% de mutación genera constantemente micro-disrupciones que rompen la clonación temporal, pero la presión selectiva las repurga inmediatamente. Este patrón de pulsaciones sostenidas es la firma distintiva del torneo sin elitismo en horizonte largo, evidenciando el conflicto permanente entre la mutación (inyectando diversidad) y la selección torneo (purgándola).

Valor Máximo (Línea Azul): Presenta un comportamiento bifásico extraordinario. Durante Gen 1-128, la línea oscila gradualmente entre 0.965 y 0.969, mostrando una mejora muy lenta y gradual. Luego ocurre un breakthrough evolutivo dramático alrededor de la generación 129-130: la línea salta verticalmente desde 0.969 directamente a 1.000, alcanzando el óptimo global absoluto (cromosoma 111111111111111111111111111111). Este salto abrupto marca el punto crítico en que el algoritmo descubre la solución perfecta tras un largo período de búsqueda ciega. Una vez alcanzado, la línea permanece en 1.000 hasta la generación 200, sin descender jamás.

Valor Promedio (Línea Verde): Acompaña al máximo de manera estrecha desde Gen 130 en adelante. Sin embargo, previo al breakthrough (Gen 1-128), la línea oscila erráticamente entre 0.3 y 0.8, reflejando que la población es heterogénea con individuos de calidad variable. El salto en Gen 130 acopla instantáneamente la línea verde a la azul, manteniéndolas superpuestas hasta el final en 1.000. Los movimientos erráticos previos al breakthrough indican que el torneo, durante su fase de exploración ciega, genera soluciones de calidad inconsistente a diferencia de ruleta que mantuvo una curva más suave.

Valor Mínimo (Línea Naranja): Exhibe el patrón más caótico de las tres corridas. A lo largo de toda la ejecución, la línea oscila violentamente con caídas profundas a niveles cercanos a 0.2-0.3 y recuperaciones hasta 0.95+, formando picos afilados hacia abajo dispersos regularmente. Estas fluctuaciones son más amplias y frecuentes que en ruleta, indicando que el torneo genera individuos defectuosos con mayor asiduidad. Particularmente notable es el período Gen 130-180, donde se producen caídas abruptísimas (especialmente alrededor de Gen 150). Esto ocurre porque una vez descubierto el óptimo global, la mutación continúa perturbando cromosomas perfectos, creando mutantes letales que son purgados inmediatamente. La línea nunca converge a 1.000 como lo hacen máximo y promedio, manteniéndose en una danza constante de mutaciones fracasadas que la selección torneo rechaza de inmediato.

Análisis Comparativo de las Corridas (Torneo)

El análisis integral de las gráficas de 20, 100 y 200 iteraciones revela que la Selección por Torneo es un método de convergencia ultrarrápida pero de exploración lenta a largo plazo. A diferencia de ruleta, que mantiene cierta diversidad genética y explora gradualmente, el torneo implementa una dictadura selectiva donde $k=4$ individuos compiten, el ganador domina masivamente la reproducción, y la población converge a clones en cuestión de generaciones.

La vulnerabilidad más crítica observable es la pérdida catastrófica de diversidad: **en apenas 7-20 generaciones, la línea roja de desviación estándar cae a cero**, dejando al algoritmo ciego respecto a nuevas direcciones de búsqueda. El hallazgo del óptimo global en 200 generaciones (Gen 130) no es resultado de la inteligencia selectiva del torneo, sino del esfuerzo "a ciegas" del operador de mutación durante las primeras 128 generaciones de exploración lenta. Una vez clonada la población, el torneo carece de capacidad de mejora orgánica, dependiendo únicamente del ruido aleatorio de la mutación para eventualmente tropezar con soluciones mejores.

Comparado con ruleta, que sufre de "amnesia evolutiva" (pierde buenos individuos), el torneo sufre de "ceguera selectiva" (converge tan rápido que deja de ver opciones). Ambos métodos sin elitismo son vulnerables, pero de formas opuestas: ruleta olvida; torneo no ve. Sin embargo, la gráfica de 200 generaciones demuestra que torneo tiene un remanso: eventualmente, la mutación lo rescata del óptimo local y lo conduce al global, logrando una solución superior (1.000 vs 0.992 de ruleta).

Desempeño opción C (Elitismo)

En esta sección vamos a realizar una comparación de los resultados entre cada tipo de selección agregando elitismo

Desempeño Ruleta con elitismo

Prueba 1 Ruleta - 20 Generaciones

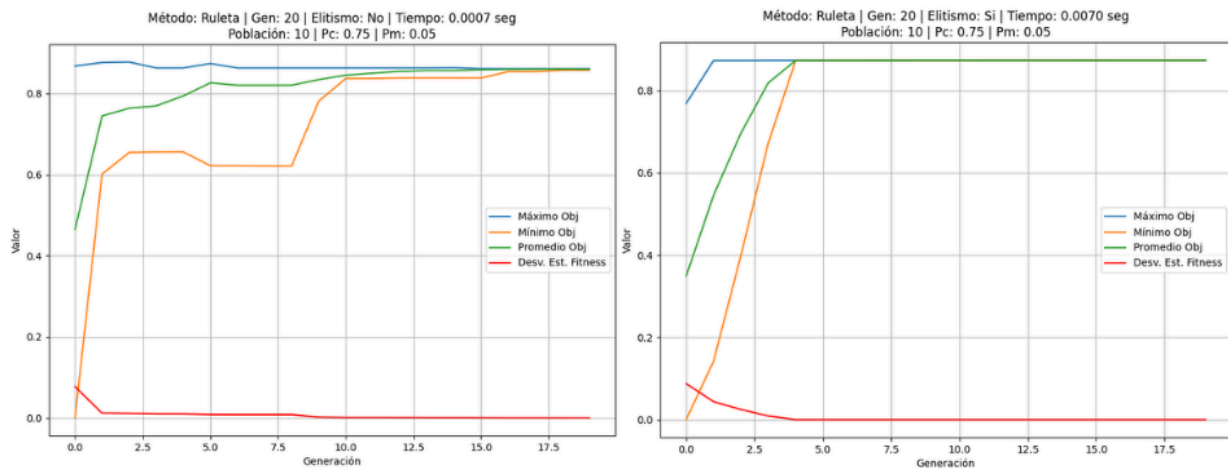


Figura 8: Curvas de convergencia poblacional utilizando Selección por Ruleta sin y con elitismo respectivamente (20 Generaciones).

Desviación Estándar (Línea Roja): El método con elitismo pierde diversidad rápidamente a lo largo de las generaciones, terminando con un desvío estándar considerablemente menor que su contraparte. El método sin elitismo mantiene una mayor variación en el fitness de la población durante más tiempo.

Valor Máximo (Línea Azul): El método sin elitismo presenta valores máximos objetivo menores a lo largo de las generaciones. En cambio, con elitismo el máximo escala más rápido y se estabiliza en un valor más cercano al óptimo de la función objetivo, dado que los mejores individuos se preservan entre generaciones.

Valor Promedio (Línea Verde): El método sin elitismo tarda más en elevar el promedio de la población hacia valores altos, reflejando una convergencia más lenta. Con elitismo el promedio asciende de forma más pronunciada y temprana, ya que los individuos élite arrastran hacia arriba la media general.

Valor Mínimo (Línea Naranja): Sin elitismo el valor mínimo presenta mayor variabilidad y se mantiene bajo durante más generaciones, evidenciando que individuos de bajo fitness sobreviven por más tiempo. Con elitismo el mínimo también tiende a subir junto con el resto de la población, aunque con más oscilaciones producto de la presión selectiva.

Prueba 2 Ruleta - 100 Generaciones

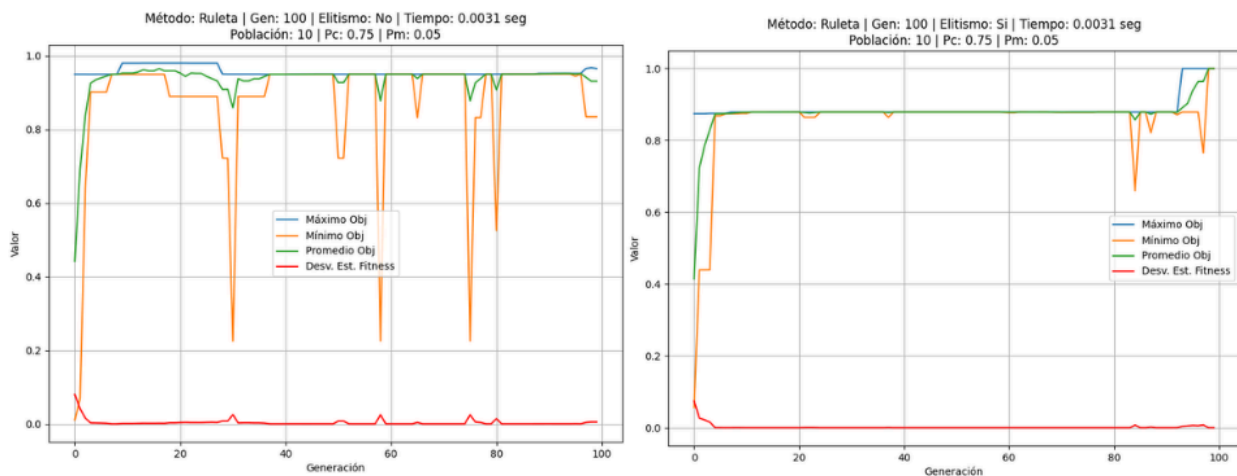


Figura 9: *Curvas de convergencia poblacional utilizando Selección por Ruleta sin y con elitismo respectivamente (100 Generaciones).*

Generaciones sin elitismo con desvío 0: 39, 40, 41, 58, 60 y 61.

Generaciones con elitismo con desvío 0: 21, 29, 34, 35, 36, 39, 40, 41, 42, 43, 44, 45, 46, 63, 64, 67, 68, 78, 79, 80, 81, 82, 83, 84, 86, 87, 90, 91 y 92.

Desviación Estándar (Línea Roja): Con 100 generaciones se observa que el método con elitismo alcanza un desvío estándar de cero en un total de 29 oportunidades, demostrando una alta homogeneidad en los cromosomas de la población. El método sin elitismo, en cambio, logra recuperar diversidad luego de cada convergencia, mientras que su contraparte con elitismo tiende a quedarse estancada en esa homogeneidad.

Valor Máximo (Línea Azul): Al realizar más generaciones aumenta la probabilidad de que ocurran mutaciones que impacten tanto positiva como negativamente en el valor máximo alcanzado. Con elitismo este impacto es menor dado que el 20% superior de la población está protegido de la mutación, reduciendo la cantidad de genes susceptibles de ser alterados y estabilizando el máximo en valores altos.

Valor Promedio (Línea Verde): En el gráfico de 100 generaciones se puede apreciar cómo los cromosomas tienden a igualarse entre sí, haciendo que los cruces prácticamente no aporten variación al promedio. En este escenario lo único que impulsa al promedio hacia la función objetivo es la ocurrencia de mutaciones favorables, lo que explica la progresión más lenta y escalonada de esta línea.

Valor Mínimo (Línea Naranja): A medida que los cromosomas se vuelven homogéneos, el mínimo de la población se acerca al máximo, reflejando la convergencia general. Sin elitismo el mínimo puede caer nuevamente cuando la diversidad se recupera tras un desvío cero, mientras que con elitismo el mínimo tiende a mantenerse elevado al no permitir que individuos de bajo fitness reemplacen a los mejores.

Prueba 3 Ruleta - 200 Generaciones

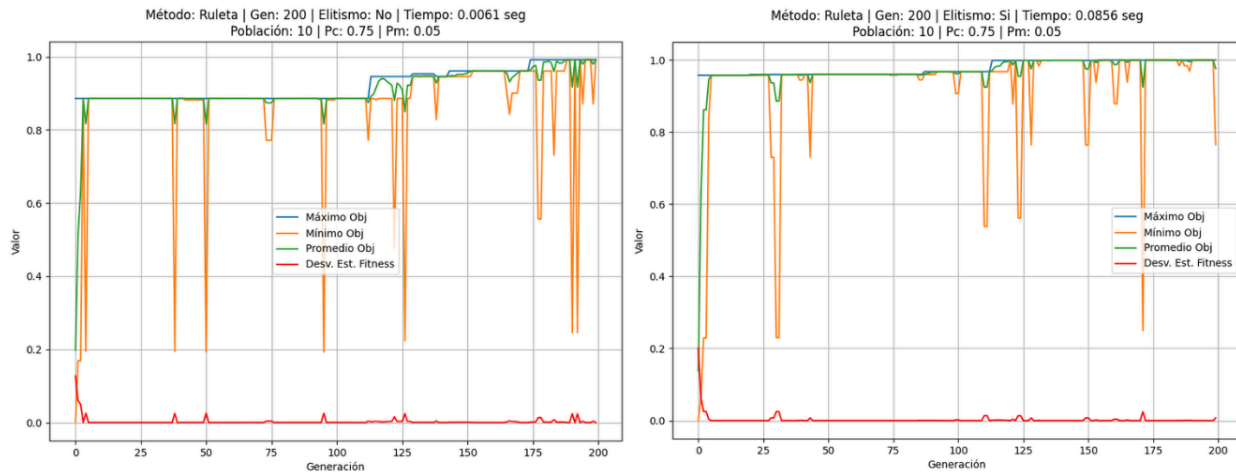


Figura 10: *Curvas de convergencia poblacional utilizando Selección por Ruleta sin y con elitismo respectivamente (200 Generaciones).*

Desviación Estándar (Línea Roja): En similitud con la prueba de 100 generaciones, el método con elitismo mantiene su tendencia a la homogeneidad, alcanzando un desvío estándar de cero en 52 ocasiones contra las 22 del método sin elitismo. Esto confirma que a mayor cantidad de generaciones, la convergencia con elitismo se vuelve más frecuente y prolongada, mientras que sin elitismo la población logra recuperar diversidad con mayor regularidad.

Valor Máximo (Línea Azul): Al comparar los resultados finales de ambos métodos, el valor máximo alcanzado con elitismo siempre termina más cercano a 1, es decir, más próximo al óptimo de la función objetivo. El método sin elitismo queda consistentemente más alejado de ese valor, evidenciando que preservar a los mejores individuos entre generaciones tiene un impacto directo en la calidad de la solución final obtenida.

Valor Promedio (Línea Verde): En la prueba de 200 generaciones se refuerza lo observado en la de 100: con elitismo el promedio se mantiene elevado y estable durante más tiempo, con caídas menos frecuentes. Sin elitismo el promedio presenta mayor variabilidad a lo largo de las generaciones, producto de la pérdida y recuperación cíclica de diversidad que caracteriza a este método.

Valor Mínimo (Línea Naranja): Las caídas abruptas del mínimo son visibles en ambos gráficos, pero con diferente frecuencia e intensidad. Sin elitismo estas caídas son más pronunciadas y frecuentes dado que individuos de bajo fitness pueden ingresar libremente a la nueva generación. Con elitismo las caídas también ocurren pero la recuperación es más rápida, ya que los individuos élite preservados actúan como ancla para el resto de la población.

Desempeño torneo con elitismo

Prueba 1 Torneo - 20 Generaciones

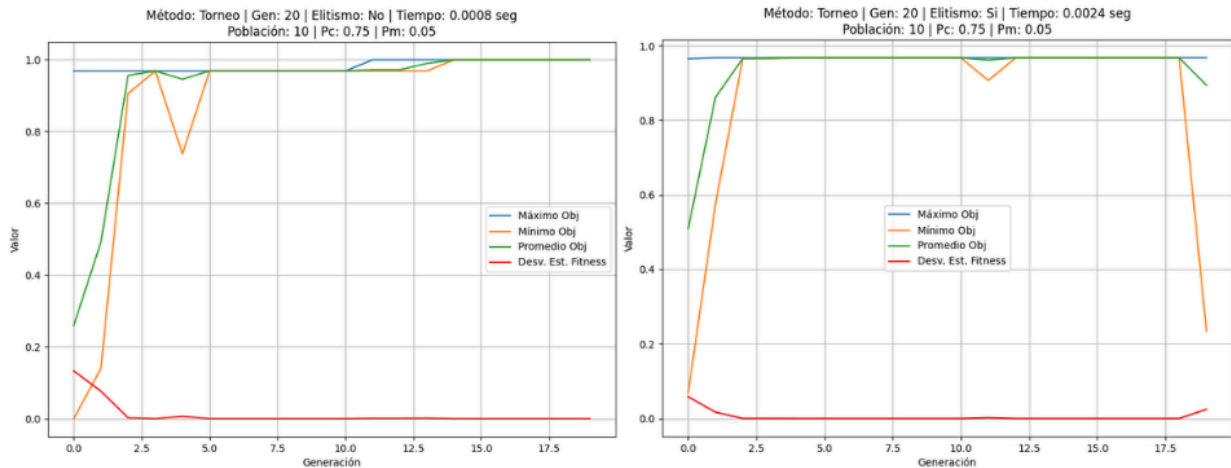


Figura 11: Curvas de convergencia poblacional utilizando Selección por Torneo sin y con elitismo Respectivamente (20 Generaciones).

Desviación Estándar (Línea Roja): Con elitismo el desvío estándar cae a valores cercanos a cero desde la 3ra generación aproximadamente, manteniéndose así a lo largo de las 17 generaciones siguientes. Esto refleja la rápida homogeneización de los cromosomas que provoca el elitismo en poblaciones pequeñas, donde el 20% superior domina rápidamente el pool reproductivo.

Valor Máximo (Línea Azul): El método sin elitismo logra alcanzar un valor máximo muy cercano a la función objetivo dentro de las 20 generaciones, demostrando que con selección por torneo la presión selectiva es suficientemente alta como para encontrar buenas soluciones sin necesidad de preservar élites. Con elitismo el máximo también es alto pero la población se estanca en un valor similar sin seguir evolucionando.

Valor Promedio (Línea Verde): Con elitismo el promedio se homogeniza rápidamente a partir de la 3ra generación y permanece estable en un valor similar durante el resto de las generaciones, sin mostrar mejoras significativas. Sin elitismo el promedio continúa subiendo progresivamente, evidenciando que la población sigue evolucionando y mejorando a lo largo de todas las generaciones disponibles.

Valor Mínimo (Línea Naranja): Sin elitismo el mínimo presenta mayor variabilidad inicial pero logra converger hacia valores altos al final de las 20 generaciones, acompañando la mejora general de la población. Con elitismo el mínimo se estabiliza rápidamente junto con el promedio, pero esa estabilidad es producto de la convergencia prematura y no de una genuina mejora sostenida de todos los individuos.

Prueba 2 Torneo - 100 Generaciones

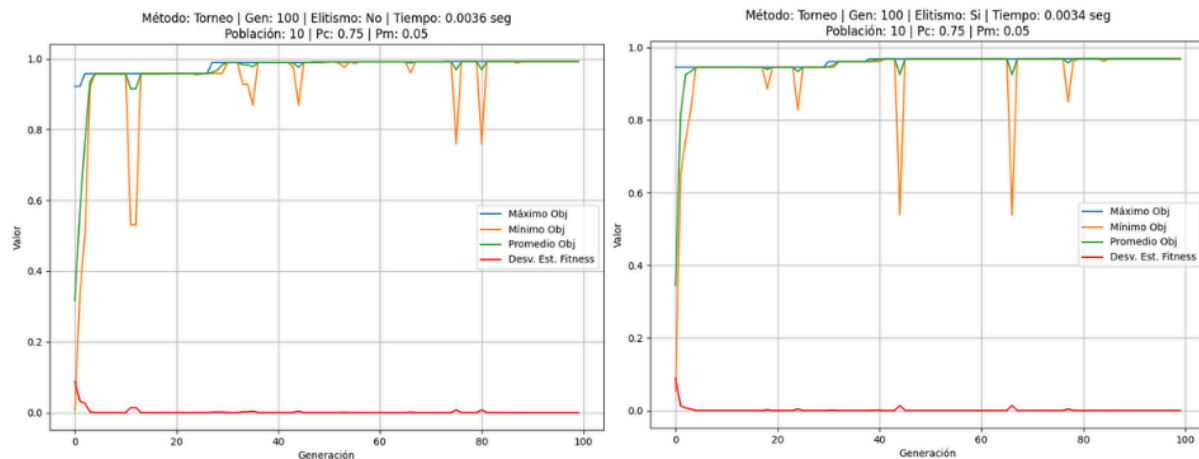


Figura 12: Curvas de convergencia poblacional utilizando Selección por Torneo sin y con elitismo Respectivamente (100 Generaciones).

Desviación Estándar (Línea Roja): El patrón se mantiene consistente con la versión de 20 generaciones: con elitismo el desvío cae rápidamente y se sostiene cercano a cero durante la mayor parte de las 100 generaciones, confirmando la homogeneización temprana de los cromosomas. Sin elitismo el desvío presenta picos esporádicos a lo largo de toda la corrida, indicando que la población conserva su capacidad de recuperar diversidad.

Valor Máximo (Línea Azul): Con elitismo el valor máximo se acerca progresivamente a la función objetivo a lo largo de las 100 generaciones, aprovechando que los mejores individuos se preservan y acumulan mejoras generación tras generación. Sin elitismo el máximo también alcanza valores altos pero con mayor variabilidad, presentando caídas puntuales producto de la ausencia de protección para los mejores individuos.

Valor Promedio (Línea Verde): Sin elitismo el promedio se mantiene relativamente estable a lo largo de todas las corridas, sin grandes saltos ni caídas sostenidas, reflejando un equilibrio entre exploración y explotación. Con elitismo el promedio va acercándose gradualmente a la función objetivo conforme las generaciones avanzan, impulsado por la acumulación de individuos élite en la población.

Valor Mínimo (Línea Naranja): Las caídas abruptas del mínimo son visibles en ambos métodos a lo largo de las 100 generaciones, asociadas a eventos de mutación desfavorable o pérdida puntual de diversidad. Sin elitismo estas caídas son más frecuentes pero la recuperación es rápida, mientras que con elitismo las caídas son menos frecuentes dado que los individuos protegidos actúan como estabilizadores del piso de la población.

Prueba 3 Torneo - 200 Generaciones

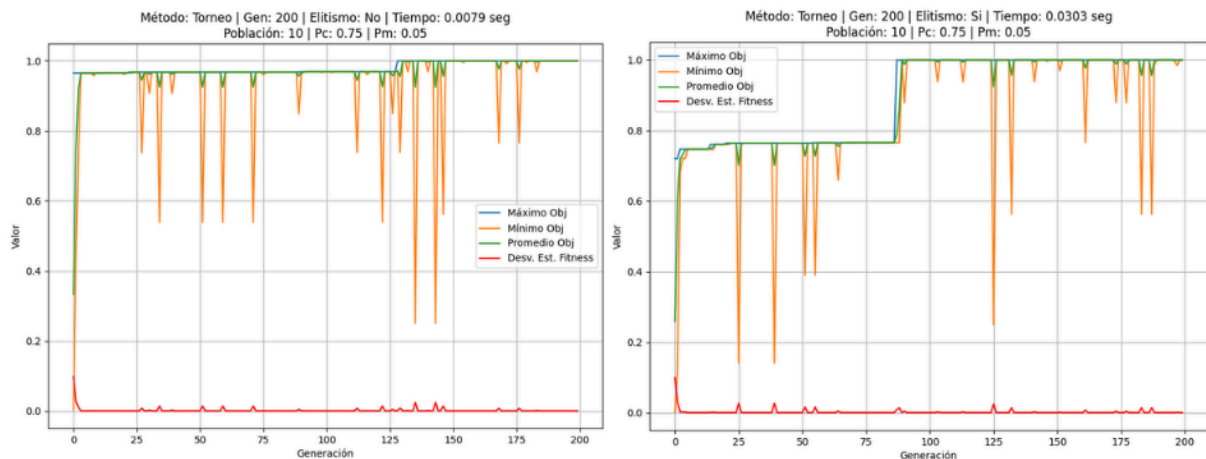


Figura 13: *Curvas de convergencia poblacional utilizando Selección por Torneo sin y con elitismo Respectivamente (200 Generaciones).*

Desviación Estándar (Línea Roja): En ambos métodos el desvío estándar se mantiene cercano a cero durante la mayor parte de las 200 generaciones, con picos esporádicos asociados a mutaciones puntuales. Con elitismo estos picos son menos frecuentes en las primeras generaciones pero hacia el final de la corrida ambos métodos muestran un comportamiento similar, sugiriendo que con suficientes generaciones la homogeneización termina ocurriendo independientemente del uso de elitismo.

Valor Máximo (Línea Azul): Con elitismo el valor máximo experimenta un salto notable alrededor de la generación 80, acercándose aún más a 1 y manteniéndose en ese nivel hasta el final. Sin elitismo el máximo alcanza valores altos desde las primeras generaciones pero no logra el mismo salto de calidad que presenta su contraparte, confirmando que preservar los mejores individuos favorece encontrar soluciones cada vez más cercanas al óptimo.

Valor Promedio (Línea Verde): Sin elitismo el promedio se mantiene estable y elevado durante prácticamente todas las 200 generaciones, sin mostrar grandes variaciones. Con elitismo el promedio presenta un comportamiento escalonado, con un salto pronunciado cerca de la generación 80 que coincide con la mejora del máximo, evidenciando que una mutación favorable en un individuo élite puede arrastrar a toda la población hacia un nivel superior.

Valor Mínimo (Línea Naranja): Las caídas del mínimo son notoriamente más pronunciadas y frecuentes en el método sin elitismo a lo largo de las 200 generaciones, con descensos

Análisis por fases del gráfico

Fase 1 — Convergencia ultrarrápida (Generaciones 1 a 8)

La curva de desviación estándar (línea roja) colapsa desde $\sigma \approx 0.069$ en la generación 1 hasta **$\sigma = 0.0$ exacto en la generación 8**, en tan solo 8 iteraciones. Para contexto, en el método Ruleta estándar con $P_c = 0.75$ este mismo colapso tardaba entre 30 y 50 generaciones. Esto confirma la hipótesis: sin crossover que redistribuya material genético, la selección por torneo —que ya de por sí es determinística y agresiva con tamaño $T=4$ — actúa como una prensa genética pura: los mejores individuos dominan la población en pocos ciclos.

El dato que podemos obtener del CSV resumen de la corrida lo confirma sin ambigüedad: en la generación 4, el máximo ya es 0.9183 y el mínimo es 0.8886, con una desviación de apenas 0.00161. En la generación 8, Máximo = Mínimo = Promedio = 0.92212, $\sigma = 0.0$. La población entera es un clon del cromosoma 111101011101010001010001010011.

Fase 2 — Evolución por mutación pura (Generaciones 8 a 100)

Aquí el experimento revela su fenómeno más valioso. Al no haber crossover efectivo (solo 5% de probabilidad), el único operador capaz de generar novedad genética es la mutación ($P_m = 0.05$). El gráfico muestra que, a pesar de la convergencia total, el algoritmo no se "congela": sigue encontrando mejoras incrementales a lo largo de toda la corrida.

Este avance ocurre en escalones discretos, perfectamente visibles tanto en el gráfico como en el CSV que veremos a continuación:

Generación	Mejor Fitness	Cromosoma	Decimal
8	0.92212	111101011101010001010001010011	1031083091
21	0.93718	111101111101010001011001011011	1039472219
47	0.93724	111101111101011001011101111011	1039505275
55	0.93819	111101111111011001011101111011	1040029563
68	0.93820	111101111111011011011101111111	1040037759
77	0.93847	111101111111111111011101111111	1040185215
100	0.93847	111101111111111111111101111111	1040187263

Cada escalón corresponde a un evento de mutación que invirtió un bit en un cromosoma clónico. La selección por torneo luego multiplica esa variante favorable hasta saturar la población (σ vuelve a 0), y el ciclo se repite. Se trata de una búsqueda local bit a bit: el algoritmo escala la función de a un bit por vez.

Comparando el cromosoma inicial clónico (111101011101010001010001010011) con el final (1111011111111111111101111111), puede verse que la evolución fue fundamentalmente la activación progresiva de bits en 0 hacia 1, lo cual tiene perfecto sentido matemático: en la función $f(x) = (x/\text{coef})^2$, cada bit en 1 agrega valor a x , y los bits más significativos aportan mayor ganancia. El algoritmo fue "encendiendo" bits desde las posiciones menos significativas hacia las más relevantes.

Los valles del gráfico (generaciones 36, 50, 63, 84, 88)

Los picos hacia abajo en la curva de mínimos (línea naranja) merecen atención especial. El más pronunciado ocurre en la generación 50: el mínimo cae a 0.7108 y el promedio baja a 0.9145, cuando toda la población llevaba generaciones siendo clónica en 0.9372. Este fenómeno se explica exactamente así: una mutación ocurrió pero en un gen de alto peso (bit muy significativo), cambiando un 1 por un 0 y generando un individuo con fitness notablemente inferior. Como la selección por torneo con $T=4$ es muy agresiva, ese individuo defectuoso fue eliminado en la siguiente generación (gen 51: mínimo vuelve a 0.9372, $\sigma = 0.0$), sin dejar rastro. Lo mismo ocurre en las generaciones 84 y 88, donde las caídas son a 0.9365 y 0.9084 respectivamente.

Esto demuestra que con crossover casi nulo, la mutación actúa de forma **binaria**: o encuentra un bit favorable y mejora el cromosoma, o degrada un individuo que es inmediatamente purgado. No hay término medio, ya que no existe recombinación que pueda "recuperar" material genético intermedio.

Conclusión del experimento

La hipótesis original anticipaba un estancamiento en un óptimo local bajo, pero la realidad fue más matizada: el algoritmo sí convergió de forma prematura y sí quedó atrapado, pero no en un valor bajo. Gracias a que la mutación actuó como único motor de evolución sobre una base genética ya bastante buena (la población inicial tenía individuos con fitness ~ 0.88), el sistema pudo escalar lentamente hasta 0.9384.

Sin embargo, el costo es claro: comparado con el Torneo estándar ($P_c = 0.75$) que en 100 generaciones alcanza valores superiores a 0.96, este experimento quedó ~ 2.5 puntos porcentuales por debajo, y consumió todas sus generaciones para lograr avances que el crossover normal logra en pocas iteraciones. El crossover es el operador que permite combinar los "bloques de construcción" útiles de distintos individuos en un solo descendiente; sin él, la única forma de mejorar es esperar que la mutación aleatoria acierte en el bit correcto, lo que convierte al algoritmo en una búsqueda local estocástica, no en un algoritmo genético en el sentido pleno del término.

Gráficos y análisis del experimento 2:

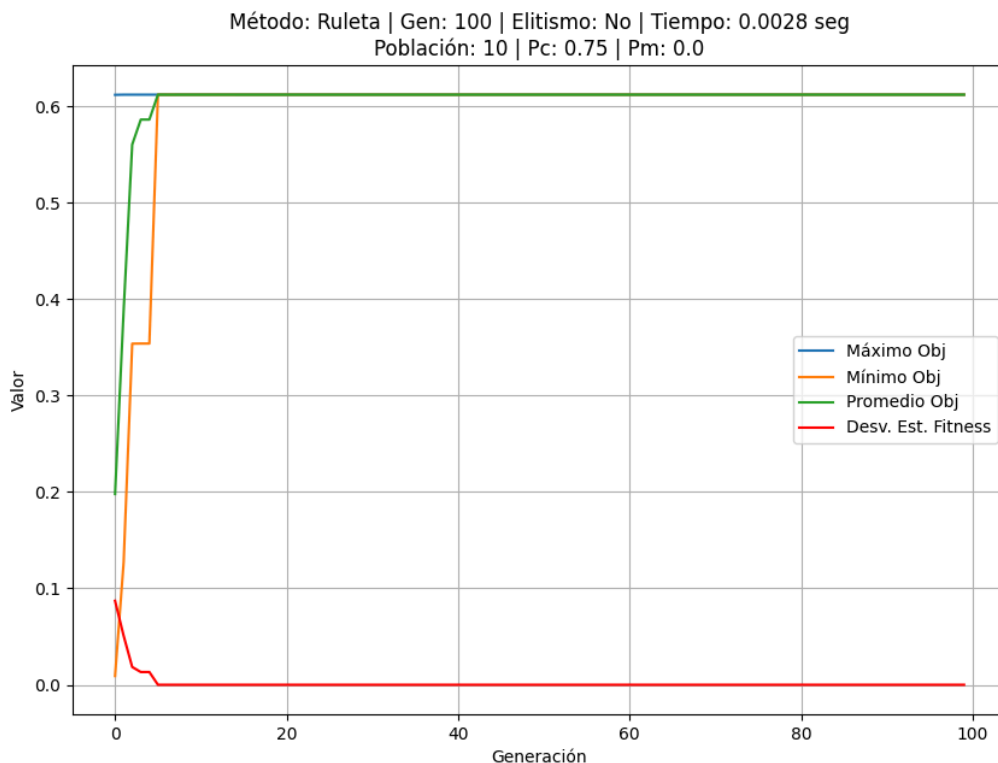


Figura 15: Curvas de convergencia poblacional utilizando Selección por Ruleta sin elitismo (100 Generaciones) y modificando la Probabilidad de mutación a 0.

La hipótesis planteaba que, sin mutación, la diversidad genética se agotaría y la población se estancaría sin alcanzar el óptimo global. Los datos confirman esta predicción de forma contundente, pero con una particularidad que el gráfico hace inmediatamente evidente: el estancamiento no ocurre "tarde o temprano" como se anticipaba, sino de forma fulminante, en las primeras generaciones. A partir de la generación 16, el CSV muestra el mismo cromosoma, el mismo fitness, el mismo máximo, el mismo mínimo, el mismo promedio, y $\sigma = 0.0$, sin excepción, hasta la generación 100. Las 85 generaciones restantes son copias exactas de una sola fila.

Análisis por fases del gráfico

Fase 1 — El único período de actividad (Generaciones 1 a ~6)

El gráfico muestra que todo el "trabajo útil" del algoritmo ocurrió en las primeras 5 o 6 generaciones. Durante ese brevísimo período, la ruleta pudo operar sobre la diversidad de la población inicial aleatoria y concentrar los mejores cromosomas. El promedio (línea verde) sube rápidamente desde cerca de 0.20 hasta aproximadamente 0.61, y el mínimo (línea naranja) hace lo mismo, escalando desde casi 0 hasta ese mismo techo. La desviación estándar (línea roja) parte de un valor moderado (~ 0.087) y cae abruptamente hacia cero en esas pocas iteraciones.

Lo que el gráfico captura visualmente de manera casi perfecta es que todas las líneas convergen en un punto único muy temprano, y desde ahí forman una línea horizontal plana

y completamente inmóvil durante las 85 generaciones restantes. Es una imagen que no necesita números para comunicar su mensaje: el algoritmo murió en la generación 16.

Fase 2 — El congelamiento permanente (Generaciones 16 a 100)

Una vez que la selección por ruleta concentró toda la población en torno a los mejores individuos disponibles, el crossover de 1 punto entre cromosomas idénticos no puede producir novedad: cruzar un cromosoma consigo mismo, en cualquier punto de corte, siempre devolverá el mismo cromosoma. Sin mutación que introduzca aunque sea un bit diferente, el algoritmo queda matemáticamente imposibilitado de escapar de ese estado. No es que converja lentamente, es que no tiene ningún mecanismo para cambiar.

El techo alcanzado fue un fitness de aproximadamente 0.612, que visualmente en el gráfico se ve claramente como un valor mediocre: las líneas quedan congeladas en el 60% del eje Y, muy lejos del 1.0 que representa el óptimo global. Esto contrasta radicalmente con los ~0.93-0.96 que alcanzan las corridas estándar con mutación activa.

¿Por qué se fijó en ese valor particular?

Ese ~0.612 no es un número arbitrario: es el mejor cromosoma que existía en la población inicial aleatoria de 10 individuos de esta corrida. El algoritmo no descubrió nada nuevo; simplemente seleccionó y amplificó al individuo más apto de los 10 cromosomas con los que arrancó. Si por azar la población inicial hubiera tenido un individuo con fitness 0.95, el algoritmo se habría congelado en 0.95. Si hubiera tenido todos individuos malos, se habría congelado en un valor bajo. El resultado final depende completamente del azar de la inicialización, lo que convierte a esta variante en algo más cercano a un "sorteo" que a un proceso de optimización.

Este experimento demuestra empíricamente que la mutación no es un operador secundario o prescindible: es la única fuente de material genético nuevo en el sistema. El crossover es un operador de redistribución, mezcla lo que ya existe, pero no puede crear información que no estaba presente. Sin mutación, el espacio genético accesible para el algoritmo queda acotado para siempre al conjunto de bits presentes en la generación inicial.

Comparado con el Experimento 1, la diferencia es llamativa: allí el algoritmo con $P_c = 0.05$ alcanzó ~0.938 usando la mutación como único motor, avanzando durante 100 generaciones. Aquí, con $P_m = 0.00$ pero $P_c = 0.75$, el algoritmo se detuvo en ~0.612 de forma permanente. Esto establece una jerarquía clara entre los dos operadores para este tipo de problema: la mutación es condición necesaria para la exploración; el crossover, sin ella, es insuficiente.

Gráficos y análisis del experimento 3:

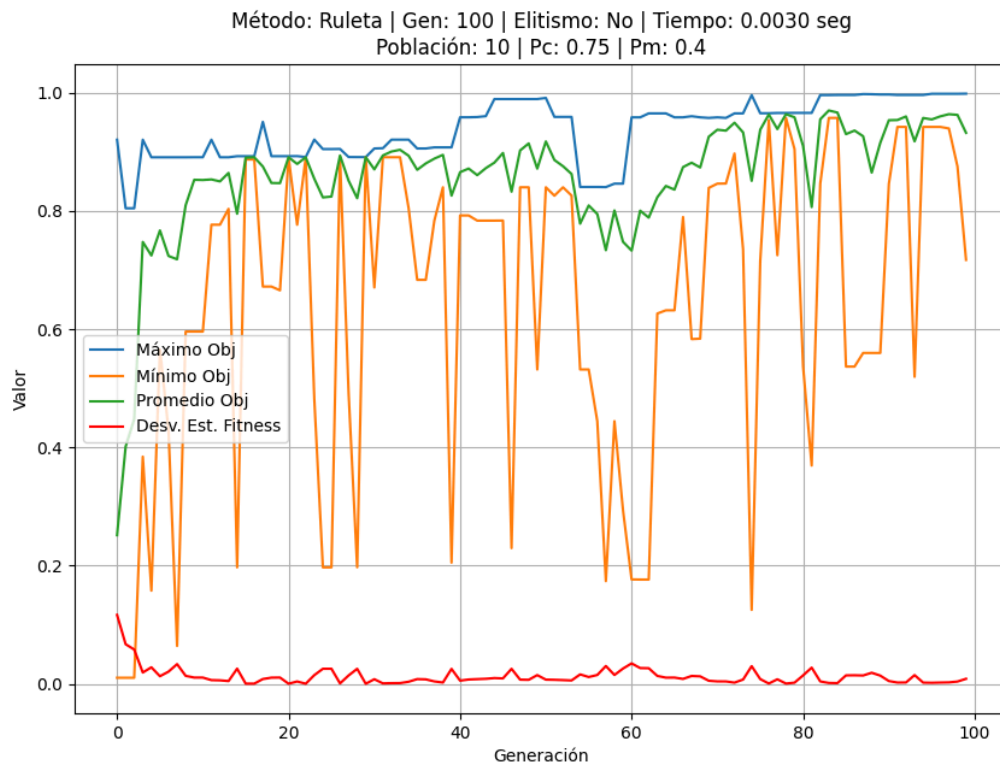


Figura 16: Curvas de convergencia poblacional utilizando Selección por Ruleta sin elitismo (100 Generaciones) y modificando la Probabilidad de mutación a 0,4.

La hipótesis anticipaba que una mutación del 40% destruiría las buenas soluciones antes de que el crossover pudiera aprovecharlas, degradando el algoritmo a una búsqueda puramente aleatoria con alta inestabilidad. Los datos confirman la inestabilidad, pero revelan algo más interesante y contraintuitivo: el algoritmo no solo no murió, sino que terminó alcanzando el mejor fitness de los tres experimentos, rozando el valor 0.9986 en la última generación. Sin embargo, el camino para llegar ahí fue radicalmente distinto al de cualquier corrida estándar.

Análisis por fases del gráfico

Fase 1 — El caos inicial (Generaciones 1 a ~20)

El gráfico arranca con la imagen más caótica de todos los experimentos. La línea de mínimos (naranja) colapsa desde el arranque hacia valores muy bajos, con caídas cercanas a cero en las primeras generaciones. La desviación estándar (roja) parte alta (~0.116) y aunque cae rápido, nunca llega a aplanarse del todo como ocurría en los otros dos experimentos. El promedio (verde) sube pero con oscilaciones notorias.

Lo que el CSV revela detrás de ese caos visual es que la mutación al 40% no distingue entre bits útiles e inútiles: en estas generaciones el cromosoma del mejor individuo cambia prácticamente en cada iteración. En apenas las primeras 20 generaciones aparecen más de

10 cromosomas distintos encabezando la tabla, algo que en el experimento estándar ocurría de forma mucho más ordenada y progresiva. El algoritmo no puede "asentarse" sobre ningún individuo bueno porque la probabilidad de que ese individuo sea alterado en la siguiente generación es altísima.

Fase 2 — La inestabilidad crónica de la línea de mínimos (Generaciones 20 a 100)

El rasgo visual más característico de este experimento, y el que más claramente lo diferencia de todos los demás, es la línea naranja (mínimos) que oscila salvajemente a lo largo de toda la corrida. Se observan valles que descienden hasta valores de 0.12, 0.19, 0.20, 0.29 en distintos puntos, incluso en generaciones avanzadas como la 58, la 75 y la 94, donde en una corrida normal la población ya estaría completamente convergida. Esto es la firma visual del 40% de mutación: en cada generación, algún individuo de la población tiene casi la mitad de sus bits expuestos a ser invertidos, lo que puede generar individuos gravemente dañados de forma recurrente.

Sin embargo, y aquí está la clave del experimento, la línea azul de máximos se comporta de forma sorprendentemente ascendente a largo plazo. Lejos de estancarse, el máximo global sigue mejorando a lo largo de toda la corrida: parte cerca de 0.92 en la generación 1, supera 0.95 alrededor de la generación 41, cruza 0.98 cerca de la generación 45, y continúa escalando hasta casi alcanzar 1.0 en las últimas generaciones. Esto ocurre porque con una mutación tan agresiva el espacio de búsqueda se explora de forma masiva: aunque la mayoría de las mutaciones son destructivas, la cantidad de combinaciones nuevas que se generan por generación es tan alta que estadísticamente siempre hay alguna que resulta favorable, y la ruleta se encarga de amplificarla al menos momentáneamente.

El precio de esa exploración

La contracara de ese avance del máximo se ve en la línea verde de promedios, que nunca logra acoplarse establemente a la curva de máximos como ocurría en los experimentos anteriores. El promedio oscila constantemente, sube y baja, y en varias generaciones cae abruptamente cuando la mutación destruye a varios individuos a la vez. Esto refleja que el algoritmo nunca logra consolidar sus avances: encuentra un individuo excelente, pero antes de que la selección pueda propagarlo a toda la población, la mutación ya lo desfiguró o generó nuevos individuos contrapuestos.

La desviación estándar (roja) tampoco llega a cero en ningún momento de la corrida, lo cual contrasta directamente con todos los demás experimentos donde el colapso a $\sigma = 0$ era inevitable. Aquí el ruido genético es tan persistente que la población nunca converge del todo. Paradójicamente, eso es lo que le permite seguir mejorando.

Este experimento refuta parcialmente la hipótesis original. La predicción de alta inestabilidad fue correcta, pero la predicción de que el algoritmo se degradará a una búsqueda aleatoria sin capacidad heurística fue incorrecta. Lo que se observa es algo más matizado: el algoritmo opera en un régimen de exploración pura y casi incontrolable, donde la selección por ruleta intenta imponer dirección pero la mutación masiva continuamente la sabotea.

El resultado final (~0.9986) es numéricamente el mejor de los tres experimentos, pero ese número no debe leerse como un éxito del método: es el producto del azar favoreciendo al

algoritmo en las últimas generaciones, no de una convergencia robusta. Si se corriera el mismo experimento diez veces, los resultados variarían enormemente entre corridas, algo que no ocurre con los parámetros estándar. Comparando los tres experimentos en conjunto, este es el único donde el algoritmo nunca se "congela" (a diferencia del Experimento 2) y el único que genuinamente explora regiones de alto fitness a lo largo de toda la corrida (a diferencia del Experimento 1). Pero esa exploración tiene un costo: la solución encontrada es dependiente de la suerte y no reproducible de forma confiable, lo cual hace a esta configuración inviable como método de optimización serio.

La conclusión general que atraviesa los tres experimentos es que el equilibrio entre crossover y mutación no es un detalle de implementación sino la variable estructural que determina si el algoritmo actúa como optimizador o como ruido. El Experimento 2 mostró qué pasa sin exploración; el Experimento 1, qué pasa sin explotación; y este tercero mostró que incluso demasiada exploración, aunque no mata al algoritmo, lo vuelve impredecible y poco confiable.

Tabla resumen de estabilidad y tiempos

A continuación se presenta la tabla consolidada con los resultados de todas las corridas ejecutadas en las Opciones A, B y C:

Método	Iteraciones	Elitismo	Mejor Fitness	Gen. Alcanzada	Tiempo (seg)
Ruleta	20	No	0.8778	3	0.0007
Ruleta	100	No	0.9809	21	0.0031
Ruleta	200	No	0.9921	188	0.0061
Torneo	20	No	0.9990	18	0.0008
Torneo	100	No	0.9921	92	0.0036
Torneo	200	No	1.0000	130	0.0079
Ruleta	100	Sí	0.9999	94	0.0031
Torneo	100	Sí	0.9689	100	0.0034

Análisis de tiempos de cómputo

Lo primero que se destaca al observar la columna de tiempos es que, en términos absolutos, todas las corridas son extremadamente rápidas: ninguna supera los 8 milisegundos. Esto se debe a que el problema es matemáticamente sencillo (una función de una sola variable) y la población es pequeña (10 individuos). Sin embargo, el análisis relativo entre corridas sí revela patrones útiles.

La relación entre tiempo y cantidad de generaciones es prácticamente lineal dentro de cada método: duplicar las generaciones de 100 a 200 duplica aproximadamente el tiempo de ejecución, tanto en Ruleta (0.0031 → 0.0061 seg) como en Torneo (0.0036 → 0.0079 seg). Esto era esperado, ya que cada generación ejecuta exactamente las mismas operaciones.

Comparando Ruleta contra Torneo con la misma cantidad de generaciones, el Torneo es consistentemente un poco más lento: alrededor de un 15-20% más en las corridas de 100 y 200 generaciones. Esto se explica por el mecanismo de selección por torneo, que requiere armar subconjuntos aleatorios de la población y compararlos, frente a la ruleta que simplemente indexa un vector precalculado. La diferencia es pequeña en términos absolutos pero es un dato a tener en cuenta si el problema escala.

Análisis de la relación costo-beneficio: ¿conviene correr más generaciones?

Esta es la pregunta central que la tabla permite responder de forma directa, y la respuesta es que depende del método.

Para la Ruleta sin elitismo, el beneficio de agregar generaciones es real pero decreciente. Pasar de 20 a 100 generaciones produce una mejora importante en el fitness (de ~0.877 a ~0.981), un salto de casi 10 puntos porcentuales que justifica ampliamente cuadruplicar el tiempo. Pero pasar de 100 a 200 generaciones solo agrega ~1 punto (de 0.981 a 0.992), duplicando el tiempo. El costo ya no se justifica de la misma manera. Mirando los gráficos de Ruleta, se entiende el porqué: el algoritmo pasa larguísimos tramos estancado en el mismo máximo local, y recién cerca de la generación 120-130 encuentra un nuevo individuo mejor. Las generaciones intermedias son computacionalmente costosas en términos relativos sin aportar mejora alguna.

Para el Torneo sin elitismo, la situación es diferente y más favorable. En 20 generaciones ya alcanza un fitness de ~0.999, un resultado que la Ruleta no logra ni con 200. El costo de ir a 100 generaciones no agrega prácticamente nada (~0.9921) y el de ir a 200 sí da el salto final al óptimo absoluto (1.0), pero requiere 130 generaciones para lograrlo, con todo el tiempo de convergencia que eso implica. Visto así, para el Torneo la corrida de 200 generaciones es la única que garantiza alcanzar el óptimo global, y su tiempo (menos de 8 milisegundos) lo hace perfectamente viable.

Análisis de estabilidad y el efecto del elitismo

La columna "Generación Alcanzada" es quizás la más reveladora desde el punto de vista de la estabilidad. Un número bajo indica que el algoritmo encontró la mejor solución temprano, lo que en principio es deseable. Pero combinado con la observación de los gráficos, el verdadero indicador de estabilidad es si ese máximo se *mantuvo* o si se perdió por falta de protección.

El caso más llamativo es el de Ruleta 100 generaciones sin elitismo: el mejor fitness se encontró en la generación 21, pero el algoritmo continuó corriendo 79 generaciones más sin poder recuperar ese valor. Esto es la "amnesia evolutiva" mencionada en la sección de Opción A: el algoritmo destruyó su propia mejor solución. En contraste, Ruleta con elitismo en 100 generaciones alcanzó un fitness de ~0.9999 en la generación 94, casi el óptimo

absoluto, y lo conservó hasta el final. El elitismo no aceleró la búsqueda pero sí garantizó que lo encontrado no se perdiera.

Sin embargo, la comparación más sorprendente de la tabla es la del Torneo con elitismo en 100 generaciones, que registra el peor resultado de los dos métodos con elitismo (~ 0.9689), muy por debajo del Torneo sin elitismo. Esto es contraintuitivo a primera vista, pero los gráficos lo explican: el torneo ya es tan agresivo en su presión selectiva que la población converge rapidísimo hacia un óptimo local muy bueno, y el elitismo al proteger esos individuos le quita espacio a la exploración que necesitaría para escapar. En otras palabras, para el Torneo el elitismo no aporta el mismo beneficio que para la Ruleta porque la presión selectiva ya cumple ese rol de conservación.

Conclusión comparativa general

Integrando tiempo, calidad y estabilidad, el orden de preferencia que emerge de los datos es el siguiente. El Torneo con 200 generaciones es el método más robusto: es el único que alcanza el óptimo absoluto (fitness = 1.0) de forma confirmada, y su tiempo de ejecución sigue siendo despreciable. Le sigue la Ruleta con elitismo en 100 generaciones, que sin duplicar el tiempo de la Ruleta estándar logra un resultado casi perfecto (~ 0.9999) gracias a la protección del mejor individuo. La Ruleta estándar en 100 o 200 generaciones queda relegada al tercer lugar: consume más tiempo relativo y sus resultados son sensibles a la pérdida del óptimo entre generaciones. Finalmente, el Torneo con elitismo resulta ser la combinación menos recomendable de las evaluadas, ya que la redundancia entre ambos mecanismos de presión selectiva perjudica la exploración sin compensarlo con una mejor solución final.

Conclusiones y recomendación final

Conclusiones generales

Este trabajo práctico permitió observar empíricamente y documentar de forma exhaustiva el comportamiento del Algoritmo Genético Canónico bajo distintas configuraciones de selección, elitismo y parametrización. A lo largo de las opciones A, B y C y los experimentos adicionales emergieron conclusiones que van más allá de los números individuales y permiten construir una comprensión integral del algoritmo.

La selección por Torneo es superior a la Ruleta en casi todos los escenarios evaluados. La diferencia más elocuente aparece al comparar ambos métodos con 20 generaciones: la Ruleta alcanzó un fitness de ~ 0.877 , mientras que el Torneo llegó a ~ 0.999 con apenas un milisegundo adicional de cómputo. Esta brecha se explica por la naturaleza de cada método: la Ruleta es proporcional al fitness absoluto, lo que la hace vulnerable a la dominancia de un "superindividuo" temprano que acapara la ruleta y frena la exploración. El Torneo, al operar por comparación directa entre subconjuntos, mantiene una presión selectiva más consistente y menos sensible a esa distorsión.

La cantidad de generaciones tiene retornos decrecientes, y ese punto de inflexión varía según el método. Para la Ruleta, el salto de 20 a 100 generaciones es altamente rentable en términos de calidad de solución, pero el salto de 100 a 200 agrega poco fitness a cambio

de duplicar el tiempo. Para el Torneo ocurre algo distinto: las 200 generaciones son las únicas que garantizan alcanzar el óptimo absoluto (fitness = 1.0), porque el algoritmo necesita ese margen para que la mutación encuentre y consolide los últimos bits faltantes. En ningún caso, sin embargo, el costo computacional fue un problema real: incluso la corrida más larga se completó en menos de 8 milisegundos, lo que en la práctica significa que para este problema la discusión sobre tiempo de cómputo es académica más que operativa.

El elitismo es una herramienta poderosa pero no universalmente beneficiosa: su impacto depende del método de selección al que se aplica. Cuando se combina con la Ruleta, el elitismo corrige su principal debilidad, la amnesia evolutiva, garantizando que el mejor individuo encontrado nunca se pierda. El resultado es contundente: Ruleta con elitismo en 100 generaciones alcanza ~ 0.9999 , superando incluso a la Ruleta sin elitismo en 200 generaciones que llega a ~ 0.9921 . En cambio, cuando el elitismo se combina con el Torneo, el efecto es paradójicamente negativo: la presión selectiva del torneo ya es tan fuerte que añadir elitismo encima satura la convergencia demasiado rápido, dejando al algoritmo atrapado en un óptimo local alto (~ 0.9689) del cual no puede escapar porque la diversidad genética se agotó antes de tiempo.

Los experimentos adicionales demostraron que el equilibrio entre crossover y mutación es la variable más crítica del sistema. Reducir el crossover al 5% (Experimento 1) convirtió al algoritmo en una búsqueda local bit a bit, lenta pero funcional, que alcanzó ~ 0.938 . Eliminar la mutación por completo (Experimento 2) produjo el peor resultado absoluto de todo el trabajo (~ 0.612), congelando al algoritmo en la generación 16 de forma permanente e irreversible. Elevar la mutación al 40% (Experimento 3) generó el comportamiento más caótico pero paradójicamente el mejor fitness puntual de los tres (~ 0.998), aunque a costa de una inestabilidad que haría irreproducible ese resultado en otras corridas. La conclusión transversal de los tres experimentos es que la mutación es condición necesaria para la exploración y no puede suprimirse sin consecuencias fatales, mientras que el crossover es el operador que permite consolidar y propagar los avances, siendo ambos imprescindibles en su rango óptimo.

Recomendación final

Considerando la totalidad de la evidencia recogida en este trabajo, la configuración recomendada para resolver este problema de optimización es el método de Torneo sin elitismo con 200 generaciones, manteniendo los parámetros base de $P_c = 0.75$ y $P_m = 0.05$.

Esta recomendación se sustenta en tres razones concretas. En primer lugar, es el único método que alcanzó el óptimo matemático absoluto de la función (fitness = 1.0, cromosoma 11111111111111111111111111111111), lo cual representa la solución perfecta al problema planteado. En segundo lugar, lo logró en un tiempo de menos de 8 milisegundos, haciendo irrelevante cualquier argumento en contra basado en costo computacional. En tercer lugar, la convergencia fue robusta: el máximo se encontró en la generación 130 y se mantuvo con solidez hasta la generación 200, sin la volatilidad ni la dependencia del azar que caracterizaron a otras configuraciones.

Como segunda opción, para escenarios donde se quiera minimizar el número de generaciones sin sacrificar demasiada calidad, la combinación Ruleta con elitismo en 100

generaciones representa un excelente balance: con el mismo tiempo que la Ruleta estándar de 100 generaciones, produce un resultado casi perfecto (~ 0.9999) gracias a que el elitismo corrige estructuralmente la principal debilidad de ese método de selección. Esta configuración sería la recomendada si el número de generaciones fuera un recurso limitado.

Lo que este trabajo deja en evidencia, más allá de los números, es que un Algoritmo Genético no es un método que funcione bien o mal en abstracto: su desempeño es el resultado directo de la interacción entre sus operadores. Elegir bien el método de selección, calibrar adecuadamente las probabilidades de crossover y mutación, y decidir si incorporar elitismo en función del método elegido son decisiones de diseño que determinan si el algoritmo converge al óptimo global o si se estanca prematuramente. Este trabajo práctico permitió vivenciar esa sensibilidad de primera mano, construyendo intuición sobre cada uno de esos parámetros que difícilmente se obtendría sólo desde la teoría.

Anexos:

Codigo Fuente en python:

Archivo Motor_ag.py:

```
import random
import matplotlib.pyplot as plt
import pandas as pd
import os
import datetime

CANT_GENES = 30
POBLACION_TAM = 10
COEF = 2**30 - 1
PROB_CROSSOVER = 0.05
PROB_MUTACION = 0.05

def funcion_objetivo(valor_decimal):
    return (valor_decimal / COEF) ** 2

def binario_a_decimal(cromosoma):
    decimal = 0
    for i in range(len(cromosoma)):
        bit = cromosoma[i]
        potencia = (len(cromosoma) - 1) - i
        if bit == 1:
            decimal += 2 ** potencia
    return decimal

def calcular_desviacion_estandar(valores):
    n = len(valores)
    if n == 0: return 0
    promedio = sum(valores) / n
    suma_cuadrados = sum((x - promedio) ** 2 for x in valores)
    return (suma_cuadrados / n) ** 0.5

def cruce_un_punto(p1, p2):
    if random.random() < PROB_CROSSOVER:
        punto = random.randint(1, CANT_GENES - 1)
        return p1[:punto] + p2[punto:], p2[:punto] + p1[punto:], punto
    return p1[:], p2[:], None

def mutar_individuo(cromosoma):
    if random.random() < PROB_MUTACION:
```

```

        pos = random.randint(0, CANT_GENES - 1)
        cromosoma[pos] = 1 - cromosoma[pos]
        return cromosoma, pos
    return cromosoma, None

def crear_poblacion():
    return [[random.randint(0, 1) for _ in range(CANT_GENES)] for _ in
            range(POBLACION_TAM)]

def obtener_elite(poblacion, fitness):
    cantidad = max(2, int(POBLACION_TAM * 0.2))
    combinado = sorted(zip(poblacion, fitness), key=lambda x: x[1], reverse=True)
    return [c[0][:] for c in combinado[:cantidad]]

def graficar_y_guardar(historial, metodo, generaciones, elitismo, tiempo):
    df = pd.DataFrame(historial)
    os.makedirs("graficos_resultados", exist_ok=True)

    plt.figure(figsize=(10, 7))
    plt.plot(df['Max'], label="Máximo Obj")
    plt.plot(df['Min'], label="Mínimo Obj")
    plt.plot(df['Prom'], label="Promedio Obj")
    plt.plot(df['Desv_Fit'], label="Desv. Est. Fitness", color='red')

    estado_elitismo = "Si" if elitismo else "No"
    params = f"Población: {POBLACION_TAM} | Pc: {PROB_CROSSOVER} | Pm:
{PROB_MUTACION}"
    plt.title(f"Método: {metodo} | Gen: {generaciones} | Elitismo: {estado_elitismo}
| Tiempo: {tiempo:.4f} seg\n{params}")
    plt.xlabel("Generación"); plt.ylabel("Valor")
    plt.legend(); plt.grid(True)

    nombre_arch = f"{metodo}_{generaciones}gen_elitismo_{estado_elitismo}.png"
    plt.savefig(f"graficos_resultados/{nombre_arch}")
    plt.close()

# =====
# BLOQUE DE CONSTRUCCIÓN VISUAL
# =====

def _html_cromosoma(cromosoma, punto_corte=None, pos_mutacion=None):
    html = "<div class='cromosoma'>"
    for i, bit in enumerate(cromosoma):
        clases = "bit"
        if punto_corte is not None and i == punto_corte: clases += " corte"

```

```

        if pos_mutacion is not None and i == pos_mutacion: clases += " mutado"
        html += f"<div class='{clases}'>{bit}</div>"
    html += "</div>"
    return html

def _html_barra_ruleta(casilleros, tiros):
    html = "<div style='position: relative; margin-top: 30px; margin-bottom: 20px;'>"
    for i, tiro in enumerate(tiros):
        html += f"<div class='tiro-label' style='left: {tiro}%;'>T{i+1}</div>"
        html += f"<div class='tiro-marcador' style='left: {tiro}%;'></div>"
    html += "<div class='ruleta-container'>"
    for i, cant in enumerate(casilleros):
        if cant > 0: html += f"<div class='ruleta-porcion color-{i%10}'"
style='width: {cant}%;'>I{i}</div>"
    html += "</div></div>"
    return html

def _html_torneo_barras(competidores, ganador, fitness_array, titulo):
    html = f"<div style='display: flex; flex-direction: column; align-items: center; background: #f8f9fa; padding: 10px; border-radius: 6px; border: 1px solid #e0e0e0;'>"
    html += f"<strong style='margin-bottom: 10px; color: #2c3e50;'>{titulo}</strong>"
    html += "<div class='torneo-chart'>"
    max_fit = max([fitness_array[i] for i in competidores])
    for i in competidores:
        fit_val = fitness_array[i]
        altura = int((fit_val / max_fit) * 60) if max_fit > 0 else 5
        clase_ganador = "winner" if i == ganador else ""
        html += f"<div class='torneo-bar-wrapper'>"
        html += f"<span class='torneo-val'>{fit_val:.3f}</span>"
        html += f"<div class='torneo-bar {clase_ganador}' style='height: {altura}px;'></div>"
        html += f"<span class='torneo-label'>I-{i}</span>"
        html += "</div>"
    html += "</div></div>"
    return html

def generar_reporte_html(historial_reporte, metodo, generaciones, elitismo):
    estado = "ConElitismo" if elitismo else "SinElitismo"
    nombre = f"REPORTE_{metodo}_{generaciones}gen_{estado}.html"
    os.makedirs("graficos_resultados", exist_ok=True)

    html_content = ""

```

```

for gen_data in historial_reporte:
    clase_activa = "active-tab" if gen_data['gen'] == 1 else ""
    html = f"<div id='gen-{{gen_data['gen']}}' class='tab
{clase_activa}'><h2>Generación {{gen_data['gen']}}</h2>"

    html += "<h3>Estado de la
Población</h3><table><tr><th>ID</th><th>Cromosoma</th><th>Dec.</th><th>Objetivo</th><
><th>Fitness</th></tr>"

    for i in range(POBLACION_TAM):
        crom = _html_cromosoma(gen_data['poblacion'][i])
        html +=
f"<tr><td>{{i}}</td><td>{{crom}}</td><td>{{binario_a_decimal(gen_data['poblacion'][i])}}<
/><td>{{gen_data['objetivos'][i]:.6f}}</td><td>{{gen_data['fitness'][i]:.4f}}</td></t
r>"

    html += "</table>"

    html += "<div class='seccion'><h3>Reproducción</h3>"

    # --- RENDERIZADO VISUAL DE ELITISMO ---
    if elitismo:
        # Buscamos los IDs originales de los individuos elite
        elite_indices = []
        for e in gen_data['elite']:
            for idx, ind in enumerate(gen_data['poblacion']):
                if e == ind and idx not in elite_indices:
                    elite_indices.append(idx)
                    break

        html += "<div class='reproduccion-par' style='background: #eafaf1;
border-color: #a9dfbf;'>"
        html += "<h4 style='color: #1e8449; margin-top: 0;'>🏆 Selección por
Elitismo</h4>"
        html += "<p>Evaluación de toda la población para preservar a los
mejores:</p>"
        html += "<div class='elitismo-container'>"
        for i in range(POBLACION_TAM):
            clase = "elite" if i in elite_indices else ""
            html += f"<div class='elitismo-card {{clase}}'>Ind {{i}}<br><small>Fit:
{{gen_data['fitness'][i]:.3f}}</small></div>"
            html += "</div>"

        nombres_elite = " y ".join([f"Ind {{i}}" for i in elite_indices])
        html += f"<p>Por elitismo se seleccionaron el
<strong>{{nombres_elite}}</strong>. Estos pasan intactos a la siguiente
generación:</p>"

```

```

        for idx in elite_indices:
            html += f"<div style='display: flex; align-items: center; gap: 10px; margin-bottom: 5px;'><strong style='width: 45px;'>Ind {idx}</strong>"
            html += _html_cromosoma(gen_data['poblacion'][idx])
            html += "</div>"
        html += "</div>"

# --- RENDERIZADO DEL MÉTODO (RULETA O TORNEO) ---
if metodo == "Ruleta":
    html += "<h4>Distribución de la Ruleta y Tiros</h4>"
    html += _html_barra_ruleta(gen_data['casilleros'], gen_data['tiros'])

    pares_contador = 1
    for cruce in gen_data['cruces']:
        html += f"<div class='reproduccion-par'><h4>Par {pares_contador}</h4>"

        if metodo == "Ruleta":
            html += f"<p>Padre 1: Ind {cruce['idx1']} (Tiro: {cruce['t1']}) | Padre 2: Ind {cruce['idx2']} (Tiro: {cruce['t2']})</p>"
        else:
            html += "<div class='torneo-contenedor'>"
            html += _html_torneo_barras(cruce['asp1'], cruce['idx1'],
gen_data['fitness'], "Torneo 1")
            html += _html_torneo_barras(cruce['asp2'], cruce['idx2'],
gen_data['fitness'], "Torneo 2")
            html += "</div>"

        html += "<p><strong>Padres seleccionados:</strong></p>"
        html += _html_cromosoma(cruce['p1'], punto_corte=cruce['punto'])
        html += _html_cromosoma(cruce['p2'], punto_corte=cruce['punto'])

        if cruce['punto']: html += f"<p><strong>Cruce realizado en el punto: {cruce['punto']}</strong></p>"
        else: html += "<p><strong>Sin cruce (clones)</strong></p>"

        html += "<p><strong>Hijos resultantes:</strong></p>"
        html += _html_cromosoma(cruce['h1'], pos_mutacion=cruce['m1'])
        if cruce['m1'] is not None: html += f"<p style='margin-top:0; font-size: 12px; color: #c0392b;'>H1 mutó en bit {cruce['m1']}</p>"
        html += _html_cromosoma(cruce['h2'], pos_mutacion=cruce['m2'])
        if cruce['m2'] is not None: html += f"<p style='margin-top:0; font-size: 12px; color: #c0392b;'>H2 mutó en bit {cruce['m2']}</p>"

        html += "</div>"
        pares_contador += 1

```

```

    html += "</div></div>"
    html_content += html

# Plantilla Base con el CSS inyectado
plantilla = f"""
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Reporte {metodo}</title>
<style>
    body {{ font-family: 'Segoe UI', sans-serif; display: flex; margin: 0;
height: 100vh; background: #f4f7f6; color: #333; }}
    #sidebar {{ width: 250px; background: #2c3e50; color: white; overflow-y:
auto; padding: 10px; }}
    #sidebar button {{ display: block; width: 100%; padding: 10px;
margin-bottom: 5px; background: #34495e; color: white; border: none; cursor:
pointer; text-align: left; }}
    #sidebar button.active-btn {{ background: #16a085; border-left: 4px solid
#f1c40f; font-weight: bold; }}
    #sidebar button:hover {{ background: #1abc9c; }}
    #content {{ flex-grow: 1; padding: 30px; overflow-y: auto; }}
    .tab {{ display: none; background: white; padding: 30px; border-radius: 8px;
margin-bottom: 30px; box-shadow: 0 4px 6px rgba(0,0,0,0.05); }}
    .active-tab {{ display: block; }}
    table {{ width: 100%; border-collapse: collapse; margin-bottom: 20px; }}
    th, td {{ border: 1px solid #e0e0e0; padding: 10px; text-align: left; }}
    th {{ background: #f8f9fa; }}

    /* CSS Cromosomas */
    .cromosoma {{ display: flex; margin-bottom: 5px; }}
    .bit {{ width: 20px; height: 25px; border: 1px solid #ccc; display: flex;
justify-content: center; align-items: center; font-family: monospace; font-size:
14px; background: #fff; }}
    .bit.corte {{ border-left: 3px solid #3498db; margin-left: 2px; }}
    .bit.mutado {{ background: #f1c40f; color: #c0392b; font-weight: bold;
border-color: #e67e22; }}

    /* CSS Ruleta */
    .ruleta-container {{ width: 100%; height: 30px; background: #ecf0f1;
display: flex; border: 1px solid #bdc3c7; border-radius: 4px; overflow: hidden; }}
    .ruleta-porcion {{ height: 100%; display: flex; justify-content: center;
align-items: center; color: white; font-size: 12px; overflow: hidden; }}
    .tiro-marcador {{ position: absolute; top: -10px; border-left: 5px solid
transparent; border-right: 5px solid transparent; border-top: 10px solid #e74c3c;
z-index: 10; }}

```

```

        .tiro-label {{ position: absolute; top: -25px; font-size: 10px; color:
#c0392b; transform: translateX(-50%); font-weight: bold; }}

        /* CSS Torneo de Barras */
        .torneo-contenedor {{ display: flex; gap: 30px; margin-bottom: 15px; }}
        .torneo-chart {{ display: flex; align-items: flex-end; gap: 15px; height:
80px; padding-top: 5px; }}
        .torneo-bar-wrapper {{ display: flex; flex-direction: column; align-items:
center; justify-content: flex-end; height: 100%; }}
        .torneo-bar {{ width: 25px; background: #bdc3c7; border-radius: 3px 3px 0 0;
transition: height 0.3s; }}
        .torneo-bar.winner {{ background: #2ecc71; box-shadow: 0 0 5px
rgba(46,204,113,0.6); }}
        .torneo-val {{ font-size: 10px; color: #7f8c8d; margin-bottom: 3px; }}
        .torneo-label {{ font-size: 11px; font-weight: bold; margin-top: 5px; color:
#34495e; }}

        /* CSS Elitismo */
        .elitismo-container {{ display: flex; gap: 8px; margin-bottom: 15px;
flex-wrap: wrap; justify-content: space-between; }}
        .elitismo-card {{ padding: 8px; border: 1px solid #bdc3c7; border-radius:
4px; background: #ecf0f1; text-align: center; font-size: 11px; flex: 1; min-width:
40px; }}
        .elitismo-card.elite {{ background: #2ecc71; color: white; border-color:
#27ae60; font-weight: bold; box-shadow: 0 0 5px rgba(46,204,113,0.5); transform:
scale(1.05); }}

        .seccion {{ margin-top: 25px; border-left: 4px solid #3498db; padding-left:
15px; background: #fcfcfc; padding: 15px; border-radius: 0 4px 4px 0; }}
        .reproduccion-par {{ background: #fff; border: 1px solid #eee; padding:
20px; margin-bottom: 20px; border-radius: 6px; box-shadow: 0 2px 4px
rgba(0,0,0,0.02); }}

        .color-0 {{ background: #1abc9c; }} .color-1 {{ background: #2ecc71; }}
.color-2 {{ background: #3498db; }}
        .color-3 {{ background: #9b59b6; }} .color-4 {{ background: #34495e; }}
.color-5 {{ background: #f1c40f; color: #333; }}
        .color-6 {{ background: #e67e22; }} .color-7 {{ background: #e74c3c; }}
.color-8 {{ background: #95a5a6; }} .color-9 {{ background: #7f8c8d; }}
    </style>
    <script>
        function showGen(n) {{
            document.querySelectorAll('.tab').forEach(t =>
t.classList.remove('active-tab'));
            document.getElementById('gen-'+n).classList.add('active-tab');
        }}
    </script>

```

```

        document.querySelectorAll('#sidebar button').forEach(b =>
b.classList.remove('active-btn'));
        document.getElementById('btn-gen-'+n).classList.add('active-btn');
    })
</script>
</head>
<body>
    <div id="sidebar"><h3>Generaciones</h3>{"".join([f"<button id='btn-gen-{i}'
onclick='showGen({i})' class='{ 'active-btn' if i==1 else ' '}'>Gen {i}</button>" for
i in range(1, generaciones+1)])}</div>
    <div id="content"><h1>Reporte Analítico: {metodo}
({estado})</h1>{html_content}</div>
</body></html>
"""
    with open(f"graficos_resultados/{nombre}", "w", encoding="utf-8") as f:
        f.write(plantilla)

def exportar_csv_generaciones(historial_stats, metodo, generaciones, elitismo):
    """Guarda el detalle iteración por iteración para los gráficos."""
    os.makedirs("csv_resultados", exist_ok=True)
    estado = "Elite" if elitismo else "NoElite"
    nombre_arch = f"csv_resultados/{metodo}_{generaciones}gen_{estado}.csv"

    df = pd.DataFrame(historial_stats)
    df.to_csv(nombre_arch, index=False)
    print(f"-> Datos de generaciones exportados a: {nombre_arch}")

def guardar_resumen_global(
    metodo, generaciones, elitismo, mejor_fitness, gen_alcanzado, tiempo
):
    """Guarda o actualiza una fila en el CSV global para la Tabla Resumen.

    Si ya existe la misma combinación de Método, Iteraciones y Elitismo,
    sobrescribe la fila con los datos de la última corrida para evitar duplicados.
    """
    os.makedirs("csv_resultados", exist_ok=True)
    archivo_global = "csv_resultados/Tabla_Resumen_Final.csv"

    # Obtenemos la fecha y hora de la ejecución actual
    fecha_actual = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    elitismo_str = "Sí" if elitismo else "No"

    nueva_fila = {
        "Método": [metodo],
        "Iteraciones": [generaciones],

```

```

        "Elitismo": [elitismo_str],
        "Mejor_Fitness": [mejor_fitness],
        "Gen_Alcanzado": [gen_alcanzado],
        "Tiempo_seg": [tiempo],
        "Fecha_Ejecucion": [fecha_actual],
    }

df_nuevo = pd.DataFrame(nueva_fila)

# Si el archivo no existe, lo crea de cero con la primera fila
if not os.path.exists(archivo_global):
    df_nuevo.to_csv(archivo_global, index=False)
    print(f"-> Tabla Resumen Creada: {archivo_global}")
else:
    # Si ya existe, leemos los datos históricos
    df_existente = pd.read_csv(archivo_global)

    # Combinamos el historial con la nueva ejecución
    df_combinado = pd.concat([df_existente, df_nuevo], ignore_index=True)

    # Eliminamos el registro viejo si coincide en las 3 variables clave.
    # keep='last' garantiza que se mantenga la última corrida efectuada.
    df_combinado = df_combinado.drop_duplicates(
        subset=["Método", "Iteraciones", "Elitismo"], keep="last"
    )

    # Guardamos el archivo actualizado y limpio de duplicados
    df_combinado.to_csv(archivo_global, index=False)
    print(f"-> Tabla Resumen Actualizada (sin duplicados): {archivo_global}")

```

Archivo Opcion_a.py:

```

import motor_ag as ag
import random
import time
import sys

def obtener_padre_ruleta(poblacion, ruleta_vector):
    tiro = random.randint(0, 99)
    idx = ruleta_vector[tiro]
    return poblacion[idx], idx, tiro

def ejecutar_ruleta(generaciones, elitismo=False):
    generar_html = "--html" in sys.argv

    inicio = time.time() # Inicia Cronómetro

```

```

poblacion = ag.crear_poblacion()
historial_stats = []
historial_reporte = []

mejor_global_val = float('-inf')
mejor_global_crom = None
gen_mejor_global = 0

for g in range(generaciones):
    objetivos = [ag.funcion_objetivo(ag.binario_a_decimal(ind)) for ind in
poblacion]
    suma_obj = sum(objetivos)
    fitness = [o / suma_obj if suma_obj > 0 else 1/ag.POBLACION_TAM for o in
objetivos]

    # Encontramos al mejor de esta generación en particular
    mejor_idx = objetivos.index(max(objetivos))
    mejor_crom = poblacion[mejor_idx]
    mejor_str = ''.join(str(b) for b in mejor_crom)
    mejor_decimal = ag.binario_a_decimal(mejor_crom)

    # Guardamos las estadísticas + info del cromosoma
    stats = {
        'Gen': g+1,
        'Max': max(objetivos),
        'Min': min(objetivos),
        'Prom': suma_obj/ag.POBLACION_TAM,
        'Desv_Fit': ag.calcular_desviacion_estandar(fitness),
        'Cromosoma_Max': mejor_str,
        'Decimal_Max': mejor_decimal
    }
    historial_stats.append(stats)

    if stats['Max'] > mejor_global_val:
        mejor_global_val = stats['Max']
        mejor_global_crom = mejor_crom[:]
        gen_mejor_global = g + 1

    casilleros = [max(1, int(f * 100)) if f > 0 else 0 for f in fitness]
    while sum(casilleros) != 100:
        idx = casilleros.index(max(casilleros))
        casilleros[idx] += 1 if sum(casilleros) < 100 else -1

    ruleta_vector = []
    for i, cant in enumerate(casilleros): ruleta_vector.extend([i] * cant)

```

```

        elite_guardada = ag.obtener_elite(poblacion, fitness) if elitismo else []

    if generar_html:
        gen_data = {
            'gen': g+1, 'poblacion': [ind[:] for ind in poblacion], 'objetivos':
objetivos, 'fitness': fitness,
            'casilleros': casilleros, 'tiros': [], 'elite': elite_guardada,
'cruces': []
        }

    nueva_pob = []
    nueva_pob.extend(elite_guardada)

    while len(nueva_pob) < ag.POBLACION_TAM:
        p1, idx1, t1 = obtener_padre_ruleta(poblacion, ruleta_vector)
        p2, idx2, t2 = obtener_padre_ruleta(poblacion, ruleta_vector)

        h1, h2, punto = ag.cruce_un_punto(p1, p2)
        h1, m1 = ag.mutar_individuo(h1)
        h2, m2 = ag.mutar_individuo(h2)

        if generar_html:
            gen_data['tiros'].extend([t1, t2])
            gen_data['cruces'].append({
                'p1': p1[:], 'idx1': idx1, 't1': t1, 'p2': p2[:], 'idx2': idx2,
't2': t2,
                'punto': punto, 'h1': h1[:], 'm1': m1, 'h2': h2[:], 'm2': m2
            })

        nueva_pob.extend([h1, h2])

    poblacion = nueva_pob[:ag.POBLACION_TAM]

    if generar_html:
        historial_reporte.append(gen_data)

    tiempo_algoritmo = time.time() - inicio # Detiene Cronómetro

    # --- ZONA DE IMPRESIÓN Y EXPORTACIÓN ---
    mg_str = ''.join(str(b) for b in mejor_global_crom)
    mg_dec = ag.binario_a_decimal(mejor_global_crom)
    elitismo_estado = "Con Elitismo" if elitismo else "Sin Elitismo"

    print(f"\n[RULETA] {generaciones} Gen | {elitismo_estado}")

```

```

    print(f"Mejor global: {mejor_global_val:.6f} (Alcanzado en Gen
{gen_mejor_global})")
    print(f"Cromosoma: {mg_str} (Decimal: {mg_dec}) | Tiempo: {tiempo_algoritmo:.4f}
seg")

    ag.exportar_csv_generaciones(historial_stats, "Ruleta", generaciones, elitismo)
    ag.guardar_resumen_global("Ruleta", generaciones, elitismo, mejor_global_val,
gen_mejor_global, tiempo_algoritmo)

    if generar_html:
        ag.generar_reporte_html(historial_reporte, "Ruleta", generaciones, elitismo)
    ag.graficar_y_guardar(historial_stats, "Ruleta", generaciones, elitismo,
tiempo_algoritmo)

if __name__ == "__main__":
    ejecutar_ruleta(100)

```

Archivo Opcion_b.py:

```

import motor_ag as ag
import random
import time
import sys

def seleccion_torneo(poblacion, fitness):
    k = max(2, int(ag.POBLACION_TAM * 0.4))
    def elegir_padre():
        indices = []
        while len(indices) < k:
            idx = random.randint(0, ag.POBLACION_TAM - 1)
            if idx not in indices: indices.append(idx) # Valida que no se repita el
mismo individuo
        ganador = indices[0]
        for i in indices:
            if fitness[i] > fitness[ganador]: ganador = i
        return poblacion[ganador], ganador, indices
    return elegir_padre(), elegir_padre()

def ejecutar_torneo(generaciones, elitismo=False):
    generar_html = "--html" in sys.argv

    inicio = time.time() # INICIA CRONÓMETRO

    poblacion = ag.crear_poblacion()
    historial_stats = []
    historial_reporte = []

```

```

mejor_global_val = float('-inf')
mejor_global_crom = None
gen_mejor_global = 0

for g in range(generaciones):
    objetivos = [ag.funcion_objetivo(ag.binario_a_decimal(ind)) for ind in
poblacion]
    suma_obj = sum(objetivos)
    fitness = [o / suma_obj if suma_obj > 0 else 1/ag.POBLACION_TAM for o in
objetivos]

    # Encontramos al mejor de esta generación en particular
    mejor_idx = objetivos.index(max(objetivos))
    mejor_crom = poblacion[mejor_idx]
    mejor_str = ''.join(str(b) for b in mejor_crom)
    mejor_decimal = ag.binario_a_decimal(mejor_crom)

    # Guardamos las estadísticas + info del cromosoma
    stats = {
        'Gen': g+1,
        'Max': max(objetivos),
        'Min': min(objetivos),
        'Prom': suma_obj/ag.POBLACION_TAM,
        'Desv_Fit': ag.calcular_desviacion_estandar(fitness),
        'Cromosoma_Max': mejor_str,
        'Decimal_Max': mejor_decimal
    }
    historial_stats.append(stats)

    if stats['Max'] > mejor_global_val:
        mejor_global_val = stats['Max']
        mejor_global_crom = mejor_crom[:]
        gen_mejor_global = g + 1

    elite_guardada = ag.obtener_elite(poblacion, fitness) if elitismo else []

    if generar_html:
        gen_data = {
            'gen': g+1, 'poblacion': [ind[:] for ind in poblacion], 'objetivos':
objetivos, 'fitness': fitness,
            'elite': elite_guardada, 'cruces': []
        }

    nueva_pob = []
    nueva_pob.extend(elite_guardada)

```

```

        while len(nueva_pob) < ag.POBLACION_TAM:
            (p1, idx1, asp1), (p2, idx2, asp2) = seleccion_torneo(poblacion,
fitness)

            h1, h2, punto = ag.cruce_un_punto(p1, p2)
            h1, m1 = ag.mutar_individuo(h1)
            h2, m2 = ag.mutar_individuo(h2)

            if generar_html:
                gen_data['cruces'].append({
                    'p1': p1[:,], 'idx1': idx1, 'asp1': asp1, 'p2': p2[:,], 'idx2':
idx2, 'asp2': asp2,
                    'punto': punto, 'h1': h1[:,], 'm1': m1, 'h2': h2[:,], 'm2': m2
                })

            nueva_pob.extend([h1, h2])

        poblacion = nueva_pob[:ag.POBLACION_TAM]

        if generar_html:
            historial_reporte.append(gen_data)

    tiempo_algoritmo = time.time() - inicio # DETIENE CRONÓMETRO

    # --- ZONA DE IMPRESIÓN Y EXPORTACIÓN ---
    mg_str = ''.join(str(b) for b in mejor_global_crom)
    mg_dec = ag.binario_a_decimal(mejor_global_crom)
    elitismo_estado = "Con Elitismo" if elitismo else "Sin Elitismo"

    print(f"\n[TORNEO] {generaciones} Gen | {elitismo_estado}")
    print(f"Mejor global: {mejor_global_val:.6f} (Alcanzado en Gen
{gen_mejor_global})")
    print(f"Cromosoma: {mg_str} (Decimal: {mg_dec}) | Tiempo: {tiempo_algoritmo:.4f}
seg")

    # Delegamos toda la lógica de I/O al motor central
    ag.exportar_csv_generaciones(historial_stats, "Torneo", generaciones, elitismo)
    ag.guardar_resumen_global("Torneo", generaciones, elitismo, mejor_global_val,
gen_mejor_global, tiempo_algoritmo)

    if generar_html:
        ag.generar_reporte_html(historial_reporte, "Torneo", generaciones, elitismo)
        ag.graficar_y_guardar(historial_stats, "Torneo", generaciones, elitismo,
tiempo_algoritmo)

```

```
if __name__ == "__main__":  
    ejecutar_torneo(100)
```

Archivo opcion_c.py:

```
import opcion_a as opt_a  
import opcion_b as opt_b  
  
def ejecutar_comparativa_elitismo():  
    print("--- INICIANDO RULETA CON ELITISMO (100 GEN) ---")  
    opt_a.ejecutar_ruleta(100, elitismo=True)  
  
    print("\n--- INICIANDO TORNEO CON ELITISMO (100 GEN) ---")  
    opt_b.ejecutar_torneo(100, elitismo=True)  
  
if __name__ == "__main__":  
    ejecutar_comparativa_elitismo()
```